



SQL NOTES



Index

S No	Topic	Page No
1	Introduction to SQL	1 - 7
2	DDL Commands	8 - 11
3	DML Commands	12 - 16
4	DQL Commands	17 - 22
5	Operators	23 - 28
6	Constraints	29 - 35
7	Aggregate Functions	36 - 40
8	String Functions	41 – 46
9	Date Functions	47 – 51
10	GROUP BY and HAVING	52 – 57
11	Joins	58 – 65
12	Set Operators	66 – 71
13	Subqueries	72 – 79
14	Views	80 – 85
15	Indexes	86 – 91
16	TCL Commands	92 – 96
17	DCL Commands	97 – 101
18	Normalization	102 – 108
19	Stored Procedures	109 – 115
20	Triggers	116 - 122
21	Common Table Expressions (CTE)	123 – 129
22	Window Functions	130 – 136

UNIT 1: Introduction to SQL

Data :

Data refers to raw facts, figures, or information that can be processed and stored for future use. Data can be in the form of numbers, text, images, dates, or symbols.

For example, a student's name, age, marks, and address are all considered data.

Information :

Information is meaningful, organized, and processed output derived from the raw data is called information.

Why Do We Need Data?

Without data:

- No information can be stored.
- Analysis becomes impossible.
- Applications cannot function properly.

With data:

- Information can be stored permanently.
- Reports can be generated.
- Decision-making becomes easier.

Examples:

Roll No	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	91

Types of Data

1. Numeric Data :

Numeric data consists of numbers that are used for performing mathematical calculations and arithmetic operations. It can store whole numbers as well as decimal values.

Examples:

- 100
- 3.14

2. Character Data :

Character data consists of alphabets, words, symbols, and text values. It is used to store information that cannot be used for mathematical calculations.

Examples:

- Priya
- Python
- SQL

3. Date Data :

Date data is used to store date and time values. It helps in managing and performing operations related to dates and times.

Examples:

- 2025-01-15
- 12:30:45

Applications

- Banking systems
- Student records
- Hospital management
- E-commerce applications

Database :

A database is an organized collection of related data stored electronically in a structured manner.

A database allows users to store, retrieve, modify, and manage information efficiently.

Row : A single record in a table is called Row, Tuples and Records.

Example :

1	sam	5000
---	-----	------

Column : A vertical collection of data is called Column, Attributes, and Fields.

Example :

EmpID
1
2
3

Table : A Collection of data in rows and columns is called Table.

Example :

Roll No	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	91

Why Do We Need Databases?

Without databases:

- Data becomes scattered.
- Searching records becomes difficult.
- Data duplication increases.

With databases:

- Information is organized.
- Retrieval becomes faster.
- Data security improves.

Examples :

- Student Database
- Employee Database
- Hospital Database
- Banking Database

Example Table

Student Table:

ID	Name	Course
101	Priya	Python
102	Ram	SQL
103	Suma	Django

Advantages

- Reduced redundancy
- Faster access
- Better security
- Data consistency

DBMS (Database Management System) :

A DBMS is software that helps users create, store, retrieve, update, and manage databases efficiently.

It acts as an interface between the user and the database.

Examples of DBMS

- MySQL
- Oracle
- Microsoft Access
- SQLite
- PostgreSQL

Functions of DBMS :

1. **Data Storage** : Stores information permanently.
2. **Data Retrieval** : Retrieves required records.
3. **Data Manipulation** : Allows insertion, updation, and deletion.
4. **Security** : Protects data from unauthorized access.
5. **Backup and Recovery** : Helps recover lost data.

Advantages

- Data sharing
- Data security
- Reduced redundancy
- Concurrent access

Disadvantages

- Costly
- Requires maintenance
- Complexity

RDBMS (Relational Database Management System) :

It stores data in the form of tables consisting of rows and columns. Tables are connected through relationships using keys.

Examples :

- MySQL
- Oracle
- SQL Server
- PostgreSQL

Example :

Students Table

StudentID	Name
101	Priya
102	Sam

Courses Table

StudentID	Course
101	Python
102	SQL

StudentID creates a relationship between both tables.

Features of RDBMS

1. **Table-Based Storage** : Data is stored in tables.
2. **Relationships** : Tables are connected using primary and foreign keys.
3. **Data Integrity** : Ensures accuracy and consistency.
4. **Multi-user Support** : Multiple users can access the database simultaneously.
5. **Security** : Provides authentication and authorization.
6. **Backup and Recovery** : Protects against data loss.
7. **Transactions** : Supports COMMIT and ROLLBACK.

Advantages

1. Reduced Data Redundancy : Duplicate data is minimized.
2. Data Security : Unauthorized access can be restricted.
3. Data Sharing : Multiple users can access the same database.
4. Backup and Recovery : Lost data can be restored.
5. Data Integrity : Maintains consistency and accuracy.

Disadvantages

1. Cost : DBMS software can be expensive.
2. Complexity : Requires trained professionals.
3. Hardware Requirements : Needs powerful hardware resources.
4. Maintenance : Regular maintenance is necessary.

SQL :

SQL stands for Structured Query Language. It is used to communicate with relational databases.

MYSQL : MYSQL is an open source database product.

SQL allows users to:

- Create databases
- Create tables
- Insert records
- Retrieve records
- Update records
- Delete records

Examples of SQL Databases

- MySQL
- Oracle
- PostgreSQL
- SQL Server

Example

SELECT * FROM student;

This command retrieves all records from the student table.

Features of SQL :

1. Simple Language: Easy to understand and learn.
2. Portable : Works on different database systems.
3. Fast Query Processing: Retrieves information quickly.
4. Supports Large Databases :Can manage millions of records.
5. Security : Provides access control.
6. Standard Language : Accepted worldwide.

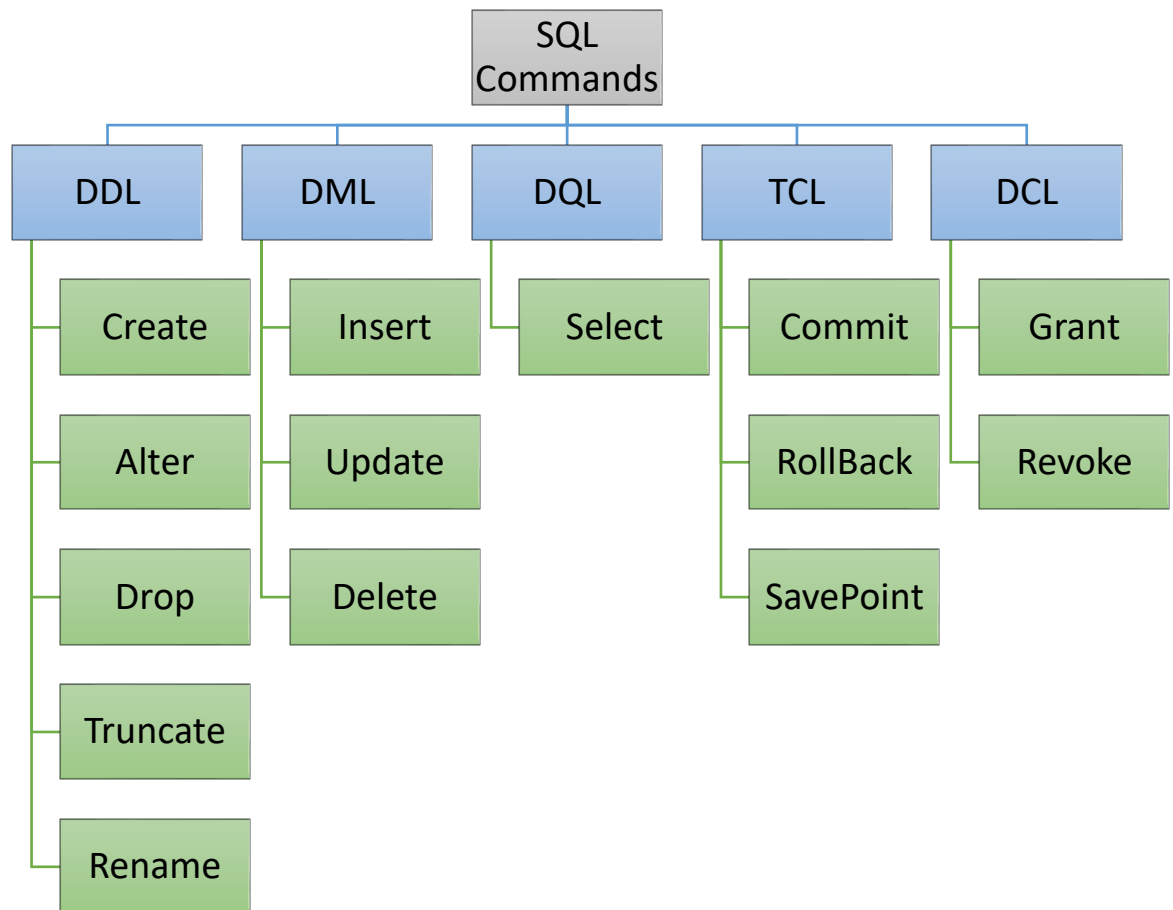
Applications of SQL :

SQL is widely used in:

- ❖ Banking Systems :Managing accounts and transactions.
- ❖ Hospital Management Systems :Storing patient information.
- ❖ Student Management Systems : Managing student records.
- ❖ E-Commerce Applications : Handling orders and products.
- ❖ Social Media Platforms : Managing user information.
- ❖ Inventory Systems : Tracking stock details.

Categories of SQL Commands :

SQL commands are divided into five categories.



1. DDL (Data Definition Language) : Used to define database structures.

Commands:

- + CREATE
- + ALTER
- + DROP
- + TRUNCATE
- + RENAME

2. DML (Data Manipulation Language) : Used to manipulate records.

Commands:

- + INSERT
- + UPDATE
- + DELETE

3. DQL (Data Query Language) : Used to retrieve information.

Command:

- + SELECT

4. TCL (Transaction Control Language) : Used for transaction management.

Commands:

- + COMMIT
- + ROLLBACK
- + SAVEPOINT

5. DCL (Data Control Language) : Used for access permissions.

Commands:

 GRANT

 REVOKE

SQL Data Types :

1. Numeric Data Types(INT) : Stores whole numbers.

Example:

marks INT

2. FLOAT : Stores decimal values.

Example:

salary FLOAT

3. DECIMAL : Stores precise decimal values.

Example:

price DECIMAL(10,2)

4. Character Data Types(CHAR) : Stores fixed-length strings.

Example:

gender CHAR(1)

5. VARCHAR : Stores variable-length strings.

Example:

name VARCHAR(50)

6. TEXT : Stores large amounts of text.

Example:

description TEXT

7. Date and Time Data Types

- DATE : Stores dates.

Example:

dob DATE

- TIME : Stores time.

Example:

start_time TIME

- DATETIME : Stores both date and time.

Example:

created_at DATETIME

UNIT 2: DDL Commands

DDL (Data Definition Language) commands are used to create, modify, and delete database objects such as databases and tables.

Commands:

1. CREATE DATABASE
2. CREATE TABLE
3. ALTER TABLE
4. DROP TABLE
5. TRUNCATE TABLE
6. RENAME TABLE

1. CREATE DATABASE :

The CREATE DATABASE command is used to create a new database. A database acts as a container that stores tables and other database objects.

Why Do We Need CREATE DATABASE?

Before creating tables and storing records, a database must first be created.

Syntax

```
CREATE DATABASE database_name;
```

Example :

```
CREATE DATABASE CollegeDB;
```

Explanation

CREATE DATABASE → SQL keyword.

CollegeDB → Name of the database.

Advantages

- Creates a new database.
- Organizes related information.
- Supports multiple tables.

2. CREATE TABLE :

The CREATE TABLE statement is used to create a new table inside a database.

A table stores information in rows and columns.

Why Do We Need CREATE TABLE?

Tables are required to store records such as employee information, student details, customer information, etc.

Syntax :

```
CREATE TABLE table_name(  
    column1 datatype,  
    column2 datatype,  
    column3 datatype  
);
```

Example :

```
CREATE TABLE Student(  
id INT,  
name VARCHAR(30),  
marks INT  
);
```

Structure Created :

id	name	marks
----	------	-------

Explanation :

- Student → Table name.
- id → Stores integer values.
- name → Stores character data.
- marks → Stores student marks.

Advantages

- Organizes data.
- Supports different datatypes.
- Makes data retrieval easier.

3. ALTER TABLE :

ALTER TABLE is used to modify an existing table.

It can:

- Add columns.
- Remove columns.
- Modify column datatypes.
- Rename columns.

Why Do We Need ALTER TABLE?

Sometimes the structure of a table needs to be changed after it has been created.

Syntax

Add Column

```
ALTER TABLE table_name
```

```
ADD column_name datatype;
```

Example

```
ALTER TABLE Student
```

```
ADD age INT;
```

Table Structure :

id	name	marks	age
----	------	-------	-----

Modify Column

```
ALTER TABLE Student
```

```
MODIFY name VARCHAR(50);
```

Drop Column

ALTER TABLE Student

DROP COLUMN age;

Advantages

- Flexible table structure.
- No need to recreate tables.
- Easy maintenance.

4. DROP TABLE :

DROP TABLE removes the entire table permanently, including both structure and records.

Why Do We Need DROP TABLE?

Used when a table is no longer required.

Syntax

DROP TABLE table_name;

Example

DROP TABLE Student;

Explanation

Both:

- Table structure
- Table data

are deleted permanently.

Advantages

- Removes unnecessary tables.
- Frees storage space.

Important Points

- Cannot be rolled back in many database systems.
- Deletes everything permanently.

5. TRUNCATE TABLE :

TRUNCATE TABLE removes all records from a table but keeps the table structure intact.

Why Do We Need TRUNCATE?

Sometimes we need to delete all records while preserving the table structure.

Syntax

TRUNCATE TABLE table_name;

Example

TRUNCATE TABLE Student;

Before

id	name	marks
101	Priya	95
102	Rahul	88

After

id	name	marks
----	------	-------

(No records)

Explanation

Only the records are removed.

The table still exists.

Advantages

- Faster than DELETE.
- Saves table structure.
- Efficient for large tables.

Difference Between DELETE and TRUNCATE

DELETE	TRUNCATE
• Removes selected rows	• Removes all rows
• WHERE clause allowed	• WHERE clause not allowed
• Slower	• Faster
• Can be rolled back	• Depends on DBMS
• Table structure remains	• Table structure remains

6. RENAME TABLE :

RENAME TABLE changes the name of an existing table.

Why Do We Need RENAME?

Sometimes table names need to be changed for better readability.

Syntax

```
RENAME TABLE old_name  
TO new_name;
```

Example

```
RENAME TABLE Student  
TO Students;
```

Explanation

Old table:

Student

becomes

Students

Advantages

- Improves readability.
- Easy maintenance.
- Better naming conventions.

UNIT 3: DML Commands

DML (Data Manipulation Language) consists of commands used to insert, modify, and delete records from a table.

DML commands work on the data stored inside tables rather than on the table structure itself.

Why Do We Need DML Commands?

Without DML commands:

- Records cannot be added.
- Existing information cannot be modified.
- Unwanted data cannot be removed.

With DML commands:

- Data can be inserted.
- Existing records can be updated.
- Unnecessary records can be deleted.

DML Commands

1. INSERT
2. UPDATE
3. DELETE

1. INSERT Statement:

The INSERT statement is used to add new records into a table.

Whenever new information needs to be stored in a database, the INSERT command is used.

Why Do We Need INSERT?

Suppose a new student joins a college. His information must be stored in the Student table.

INSERT helps us add that information.

Syntax

Insert Single Row

```
INSERT INTO table_name  
VALUES(value1, value2, value3);
```

Insert Specific Columns

```
INSERT INTO table_name(column1, column2)  
VALUES(value1, value2);
```

Example

Suppose we have the following table:

Student Table

ID	Name	Marks
----	------	-------

Insert a record:

```
INSERT INTO Student  
VALUES(101,'Priya',95);
```

Output

ID	Name	Marks
101	Priya	95

Insert Multiple Records

```
INSERT INTO Student
VALUES
(102,'Rahul',88),
(103,'Kiran',92),
(104,'Anjali',90);
```

Output

ID	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	92
104	Anjali	90

Explanation :

- INSERT INTO specifies the table.
- VALUES contains the data to be inserted.
- Each row represents one record.

Advantages

- Adds new records.
- Supports multiple insertions.
- Easy to use.

2. UPDATE Statement :

The UPDATE statement is used to modify existing records in a table. It allows changing one or more column values of already stored records.

Why Do We Need UPDATE?

Information may change over time.

Examples:

- Employee salary increases.
- Student marks are corrected.
- Customer address changes.

UPDATE helps modify such records.

Syntax :

```
UPDATE table_name
SET column_name = value
WHERE condition;
```

Example :

Student Table:

ID	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	92

Suppose Rahul's marks become 91.

```
UPDATE Student
```

```
SET marks = 91
```

```
WHERE id = 102;
```

Output

ID	Name	Marks
101	Priya	95
102	Rahul	91
103	Kiran	92

Updating Multiple Columns

```
UPDATE Student
```

```
SET name='Rohit',
```

```
marks=96
```

```
WHERE id=103;
```

Output

ID	Name	Marks
101	Priya	95
102	Rahul	91
103	Rohit	96

Explanation

- SET specifies the column values to change.
- WHERE identifies which row should be updated.

Important Point

If the WHERE clause is omitted, all rows will be updated.

Example:

```
UPDATE Student
```

```
SET marks = 100;
```

All students' marks become 100.

Advantages

- Modifies existing records.
- Saves time.
- Avoids deleting and reinserting data.

3. DELETE Statement :

The DELETE statement removes records from a table.

Unlike DROP, DELETE removes only the data and keeps the table structure intact.

Why Do We Need DELETE?

Sometimes records are no longer required.

Examples:

- Employee resigns.
- Student leaves college.
- Product becomes unavailable.

DELETE removes such records.

Syntax:

```
DELETE FROM table_name  
WHERE condition;
```

Example

Student Table:

ID	Name	Marks
101	Priya	95
102	Rahul	91
103	Rohit	96

Delete Rahul's record:

```
DELETE FROM Student  
WHERE id = 102;
```

Output

ID	Name	Marks
101	Priya	95
103	Rohit	96

Delete All Records

```
DELETE FROM Student;
```

Output

ID	Name	Marks
----	------	-------

(No records)

The table still exists.

Explanation

- DELETE removes rows.
- Table structure remains unchanged.
- WHERE clause specifies which rows to delete.

Important Point

If WHERE is omitted:

```
DELETE FROM Student;
```

All rows are deleted.

Advantages

Removes unwanted records.

Maintains data consistency.

Keeps table structure intact.

Difference Between DROP, DELETE, and TRUNCATE

Feature	DELETE	TRUNCATE	DROP
Removes Rows	Yes	Yes	Yes
Removes Table Structure	No	No	Yes
WHERE Clause Allowed	Yes	No	No
Can Delete Specific Rows	Yes	No	No
Faster	Moderate	Very Fast	Fast
Table Exists After Execution	Yes	Yes	No

Summary of DML Commands

Command	Purpose
INSERT	Adds records
UPDATE	Modifies records
DELETE	Removes records

UNIT 4: DQL Commands

DQL (Data Query Language) is a category of SQL commands used to retrieve and display data from database tables.

The main command in DQL is:

- SELECT

DQL helps users search, filter, sort, and display information stored in a database.

Why Do We Need DQL Commands?

Without DQL commands:

- Data stored in tables cannot be viewed.
- Finding specific information becomes difficult.
- Reports cannot be generated.

With DQL commands:

- Data can be retrieved easily.
- Records can be filtered.
- Information can be sorted and grouped.

DQL Commands

1. SELECT
2. DISTINCT
3. WHERE
4. ORDER BY
5. LIMIT
6. TOP Clause

1. SELECT Statement :

The SELECT statement is used to retrieve data from one or more tables. It is the most frequently used SQL command.

Why Do We Need SELECT?

Suppose we want to view student records stored in the Student table. SELECT allows us to retrieve and display those records.

Syntax

Retrieve All Columns

```
SELECT * FROM table_name;
```

Retrieve Specific Columns

```
SELECT column1, column2  
FROM table_name;
```

Example

Student Table

ID	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	91

Query:

```
SELECT * FROM Student;
```

Output

ID	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	91

Selecting Particular Columns

```
SELECT Name, Marks  
FROM Student;
```

Output

Name	Marks
Priya	95
Rahul	88
Kiran	91

Advantages

- Retrieves data quickly.
- Supports filtering and sorting.
- Generates reports.

2. DISTINCT Clause :

DISTINCT is used to remove duplicate values from the result set.

It displays only unique values.

Why Do We Need DISTINCT?

Sometimes multiple rows contain repeated values.

DISTINCT helps eliminate duplicates.

Syntax

```
SELECT DISTINCT column_name  
FROM table_name;
```

Example

Employee Table

ID	Department
1	HR
2	IT
3	HR
4	Finance
5	IT

Query:

```
SELECT DISTINCT Department  
FROM Employee;
```

Output

Department
HR
IT
Finance

Explanation

Repeated department names are displayed only once.

Advantages

- Removes duplicates.
- Produces unique results.
- Useful in analysis.

3. WHERE Clause:

The WHERE clause is used to filter records based on a condition.

Only rows satisfying the condition are returned.

Why Do We Need WHERE?

In large tables, we often need only specific records rather than all records.

Syntax

```
SELECT *  
FROM table_name  
WHERE condition;
```

Example

Student Table

ID	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	91

Query:

```
SELECT *  
FROM Student  
WHERE Marks > 90;
```

Output

ID	Name	Marks
101	Priya	95
103	Kiran	91

Explanation

Only students having marks greater than 90 are displayed.

Advantages

- Filters unnecessary data.
- Speeds up searching.
- Produces precise results.

4. ORDER BY Clause

ORDER BY is used to sort records in ascending or descending order.

Why Do We Need ORDER BY?

Data becomes easier to understand when arranged in a proper order.

Syntax

Ascending Order

```
SELECT *  
FROM table_name  
ORDER BY column_name ASC;
```

Descending Order

```
SELECT *  
FROM table_name  
ORDER BY column_name DESC;
```

Example

Student Table

ID	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	91

Query:

```
SELECT *  
FROM Student  
ORDER BY Marks DESC;
```

Output

ID	Name	Marks
101	Priya	95
103	Kiran	91
102	Rahul	88

Explanation

Records are arranged from highest marks to lowest marks.

Advantages

- Improves readability.
- Helps ranking.
- Useful in reports.

Applications

- Ranking students.
- Sorting products by price.
- Displaying salaries in descending order.

5. LIMIT Clause

LIMIT restricts the number of rows returned by a query.

Why Do We Need LIMIT?

Sometimes only a few records are required instead of the entire table.

Syntax

```
SELECT *  
FROM table_name  
LIMIT number;
```

Example

```
SELECT *  
FROM Student  
LIMIT 2;
```

Output

ID	Name	Marks
101	Priya	95
102	Rahul	88

Explanation

Only the first two records are displayed.

Advantages

- Improves performance.
- Reduces unnecessary output.
- Useful for pagination.

LIMIT with ORDER BY**Example:**

```
SELECT *  
FROM Student  
ORDER BY Marks DESC  
LIMIT 1;
```

Output

ID	Name	Marks
101	Priya	95

Explanation

Displays the student with the highest marks.

6. TOP Clause:

TOP is used to retrieve a specific number of records.

It is mainly used in SQL Server.

Syntax

```
SELECT TOP number *  
FROM table_name;
```

Example

```
SELECT TOP 2 *  
FROM Student;
```

Output

ID	Name	Marks
101	Priya	95
102	Rahul	88

TOP with ORDER BY

```
SELECT TOP 1 *  
FROM Student  
ORDER BY Marks DESC;
```

Output

ID	Name	Marks
101	Priya	95

Advantages

- Retrieves only required rows.
- Improves performance.
- Useful in reporting.

Difference Between LIMIT and TOP

LIMIT	TOP
Used in MySQL and PostgreSQL	Used in SQL Server
Written at the end of query	Written after SELECT
Retrieves specified rows	Retrieves specified rows

Summary of DQL Commands

Command	Purpose
SELECT	Retrieve data
DISTINCT	Remove duplicate values
WHERE	Filter records
ORDER BY	Sort records
LIMIT	Restrict number of rows
TOP	Retrieve top rows

UNIT 5: Operators

Operators are special symbols or keywords used to perform operations on values and expressions in SQL.

Operators are mainly used with the **WHERE clause** to filter records and perform calculations.

Why Do We Need Operators?

Without operators:

- Data cannot be filtered.
- Calculations cannot be performed.
- Comparing records becomes difficult.

With operators:

- Conditions can be applied.
- Calculations can be performed.
- Records can be filtered efficiently.

Types of SQL Operators

SQL operators are classified into four categories:

1. Arithmetic Operators
2. Comparison Operators
3. Logical Operators
4. Special Operators

1. Arithmetic Operators :

Arithmetic operators are used to perform mathematical calculations.

Addition Operator (+) :

The addition operator is used to add two values.

Syntax

```
SELECT value1 + value2;
```

Example

```
SELECT 10 + 20;
```

Output

30

Applications

- Salary calculations
- Total marks calculation

Subtraction Operator (-) :

Used to subtract one value from another.

Example

```
SELECT 50 - 20;
```

Output

30

Applications

- Profit and loss calculations
- Remaining balance calculations

Multiplication Operator (*) :

Used to multiply two values.

Example

```
SELECT 5 * 4;
```

Output

20

Applications

- Price × Quantity
- Area calculations

Division Operator (/) :

Used to divide one value by another.

Example

```
SELECT 20 / 5;
```

Output

4

Applications

- Average calculations
- Percentage calculations

2. Comparison Operators :

Comparison operators compare two values and return TRUE or FALSE. They are mostly used in the WHERE clause.

Equal To (=) :

Checks whether two values are equal.

Example

```
SELECT *  
FROM Student  
WHERE Marks = 95;
```

Output

Students having marks equal to 95.

Greater Than (>) :

Returns records having values greater than the specified value.

Example

```
SELECT *  
FROM Student  
WHERE Marks > 90;
```

Less Than (<) :

Returns records having values less than the specified value.

Example

```
SELECT *  
FROM Student  
WHERE Marks < 90;
```

 Greater Than or Equal To (>=) :

Example

```
SELECT *  
FROM Student  
WHERE Marks >= 90;
```

 Less Than or Equal To (<=) :

Example

```
SELECT *  
FROM Student  
WHERE Marks <= 90;
```

 Not Equal To (!=) :

Returns records whose values are not equal.

Example

```
SELECT *  
FROM Student  
WHERE Marks != 90;
```

Summary of Comparison Operators

Operator	Meaning
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
!=	Not equal to

3. Logical Operators :

Logical operators combine multiple conditions.
They return TRUE or FALSE.

 AND Operator :

Returns records only if all conditions are true.

Syntax

```
SELECT *  
FROM table_name  
WHERE condition1 AND condition2;
```

Example :

```
SELECT *  
FROM Student  
WHERE Marks > 80  
AND City='Hyderabad';
```

Explanation

Both conditions must be true.

 OR Operator :

Returns records if at least one condition is true.

Example

```
SELECT *  
FROM Student  
WHERE Marks > 90  
OR City='Vijayawada';
```

Explanation :

Either condition can be true.

 NOT Operator :

Reverses a condition.

Example

```
SELECT *  
FROM Student  
WHERE NOT Marks = 90;
```

Explanation

Returns all students except those having marks equal to 90.

Summary of Logical Operators

Operator	Meaning
AND	Both conditions must be true
OR	At least one condition must be true
NOT	Reverses the condition

4. Special Operators :

Special operators provide advanced filtering capabilities.

 IN Operator :

The IN operator is used to specify multiple values in a WHERE clause.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name IN(value1, value2);
```

Example :

```
SELECT *  
FROM Student  
WHERE City IN('Hyderabad','Chennai');
```

Explanation

Returns students belonging to Hyderabad or Chennai.

 BETWEEN Operator :

BETWEEN returns values within a specified range.

Example

```
SELECT *  
FROM Student  
WHERE Marks BETWEEN 80 AND 95;
```

Explanation

Returns students whose marks lie between 80 and 95.

LIKE Operator :

LIKE is used for pattern matching.

Wildcards

%

Represents multiple characters.

Example:

```
SELECT *  
FROM Student  
WHERE Name LIKE 'P%';
```

Output

```
Priya  
Praveen  
Pooja
```

_

Represents a single character.

Example

```
SELECT *  
FROM Student  
WHERE Name LIKE '_a%';
```

Applications

- Searching names
- Searching email addresses

EXISTS Operator :

EXISTS checks whether a subquery returns records.

Example

```
SELECT Name  
FROM Student  
WHERE EXISTS  
(  
  SELECT *  
  FROM Course  
  WHERE Student.id = Course.id  
);
```

Explanation

Returns records if the subquery contains rows.

ANY Operator :

ANY compares a value with any value returned by a subquery.

Example

```
SELECT *  
FROM Employee  
WHERE Salary > ANY  
(  
  SELECT Salary  
  FROM Manager  
);
```

Explanation

Returns employees whose salary is greater than at least one manager's salary.

 ALL Operator :

ALL compares a value with all values returned by a subquery.

Example

```
SELECT *  
FROM Employee  
WHERE Salary > ALL  
(  
  SELECT Salary  
  FROM Manager  
);
```

Explanation

Returns employees whose salary is greater than every manager's salary.

Summary of Special Operators

Operator	Purpose
IN	Matches multiple values
BETWEEN	Specifies a range
LIKE	Pattern matching
EXISTS	Checks subquery existence
ANY	Compares with any value
ALL	Compares with all values

UNIT 6: Constraints

Constraints are rules applied to table columns to ensure that only valid and accurate data is stored in the database.

They help maintain data integrity, consistency, and reliability. Constraints prevent users from entering incorrect or duplicate data.

Why Do We Need Constraints?

Without constraints:

- Duplicate records may occur.
- Invalid data can be stored.
- Relationships between tables cannot be maintained.
- Database consistency may be lost.

With constraints:

- Data becomes accurate.
- Duplicate values are prevented.
- Relationships between tables are maintained.
- Data integrity is improved.

Types of Constraints

SQL provides the following constraints:

1. NOT NULL
2. UNIQUE
3. PRIMARY KEY
4. FOREIGN KEY
5. CHECK
6. DEFAULT
7. AUTO_INCREMENT

1. NOT NULL Constraint :

The NOT NULL constraint ensures that a column cannot contain NULL values.

It means every record must have a value for that column.

Why Do We Need NOT NULL?

Some fields are mandatory.

For example:

- Student Name
- Employee ID
- Product Name

These values should never be empty.

Syntax

```
CREATE TABLE Student(  
  ID INT,  
  Name VARCHAR(50) NOT NULL  
);
```

Example

```
CREATE TABLE Student(  
  ID INT,  
  Name VARCHAR(50) NOT NULL,  
  Marks INT  
);
```

Valid Insert

```
INSERT INTO Student  
VALUES(101,'Priya',95);
```

Invalid Insert

```
INSERT INTO Student  
VALUES(102,NULL,90);
```

Output

Error because Name cannot be NULL.

Advantages

- Prevents missing values.
- Ensures mandatory information.
- Improves data quality.

2. UNIQUE Constraint :

The UNIQUE constraint ensures that all values in a column are different.

Duplicate values are not allowed.

Why Do We Need UNIQUE?

Some information must be unique.

Examples:

- Email IDs
- Phone numbers
- Passport numbers

Syntax

```
CREATE TABLE Employee(  
  ID INT,  
  Email VARCHAR(100) UNIQUE  
);
```

Example

```
CREATE TABLE Employee(  
  ID INT,  
  Email VARCHAR(100) UNIQUE  
);
```

Valid Inserts

```
INSERT INTO Employee  
VALUES(1,'priya@gmail.com');  
INSERT INTO Employee  
VALUES(2,'rahul@gmail.com');
```


Invalid Insert

```
INSERT INTO Employee  
VALUES(3,'priya@gmail.com');
```

Output

Error because duplicate email addresses are not allowed.

Advantages

- Prevents duplicate entries.
- Maintains uniqueness.
- Improves database accuracy.

3. PRIMARY KEY Constraint :

A PRIMARY KEY uniquely identifies each record in a table.

It is a combination of:

- UNIQUE constraint
- NOT NULL constraint

A table can have only one primary key.

Why Do We Need PRIMARY KEY?

Every record should have a unique identity.

For example:

Student IDs

Student_ID	Name
101	Priya
102	Rahul
103	Kiran

Student_ID uniquely identifies each student.

Syntax

```
CREATE TABLE Student(  
ID INT PRIMARY KEY,  
Name VARCHAR(50)  
);
```

Example

```
CREATE TABLE Student(  
ID INT PRIMARY KEY,  
Name VARCHAR(50),  
Marks INT  
);
```

Valid Inserts

```
INSERT INTO Student  
VALUES(101,'Priya',95);  
INSERT INTO Student  
VALUES(102,'Rahul',88);
```

Invalid Insert

```
INSERT INTO Student  
VALUES(101,'Kiran',91);
```

Output

Error because duplicate primary keys are not allowed.

Properties of Primary Key

- Unique values.
- Cannot contain NULL values.
- One primary key per table.
- Used to identify records uniquely.

4. FOREIGN KEY Constraint :

A FOREIGN KEY establishes a relationship between two tables.

It refers to the PRIMARY KEY of another table.

Why Do We Need FOREIGN KEY?

Relationships between tables are essential.

For example:

Department Table

DepartmentID	DepartmentName
1	HR
2	IT

Employee Table

EmployeeID	Name	DepartmentID
101	Priya	1
102	Rahul	2

DepartmentID in Employee table refers to DepartmentID in Department table.

Syntax

```
CREATE TABLE Employee(  
EmployeeID INT PRIMARY KEY,  
Name VARCHAR(50),  
DepartmentID INT,  
FOREIGN KEY (DepartmentID)  
REFERENCES Department(DepartmentID)  
);
```

Example

Parent Table

```
CREATE TABLE Department(  
DepartmentID INT PRIMARY KEY,  
DepartmentName VARCHAR(30)  
);
```

Child Table

```
CREATE TABLE Employee(  
EmployeeID INT PRIMARY KEY,  
Name VARCHAR(50),  
DepartmentID INT,  
FOREIGN KEY(DepartmentID)  
REFERENCES Department(DepartmentID)  
);
```

Advantages

- Maintains relationships.
- Prevents invalid references.
- Improves consistency.

5. CHECK Constraint :

The CHECK constraint restricts values that can be entered into a column. It allows only values satisfying a specified condition.

Why Do We Need CHECK?

To avoid invalid data.

Examples:

- Marks should be between 0 and 100.
- Age should be greater than 18.

Syntax

```
CREATE TABLE Student(  
Marks INT CHECK(Marks>=0)  
);
```

Example

```
CREATE TABLE Student(  
ID INT,  
Name VARCHAR(50),  
Marks INT CHECK(Marks BETWEEN 0 AND 100)  
);
```

Valid Insert

```
INSERT INTO Student  
VALUES(101,'Priya',95);
```

Invalid Insert

```
INSERT INTO Student  
VALUES(102,'Rahul',150);
```

Output

Error because marks cannot exceed 100.

Advantages

- Prevents invalid data.
- Improves accuracy.
- Maintains business rules.

6. DEFAULT Constraint :

The DEFAULT constraint automatically assigns a value when no value is specified.

Why Do We Need DEFAULT?

Sometimes users may not provide values.
DEFAULT provides predefined values.

Syntax

```
CREATE TABLE Employee(  
    Country VARCHAR(20)  
    DEFAULT 'India'  
);
```

Example

```
CREATE TABLE Employee(  
    ID INT,  
    Name VARCHAR(50),  
    Country VARCHAR(20)  
    DEFAULT 'India'  
);
```

Insert

```
INSERT INTO Employee(ID,Name)  
VALUES(101,'Priya');
```

Output

ID	Name	Country
101	Priya	India

Advantages

- Saves time.
- Prevents NULL values.
- Maintains consistency.

7. AUTO_INCREMENT Constraint :

AUTO_INCREMENT automatically generates sequential numbers whenever a new record is inserted.

Why Do We Need AUTO_INCREMENT?

Manually assigning IDs is difficult and error-prone.
AUTO_INCREMENT automatically generates unique IDs.

Syntax

(MySQL)

```
CREATE TABLE Student(  
    ID INT AUTO_INCREMENT,  
    Name VARCHAR(50),  
    PRIMARY KEY(ID)  
);
```

Example

```
CREATE TABLE Student(  
ID INT AUTO_INCREMENT PRIMARY KEY,  
Name VARCHAR(50)  
);
```

Insert :

```
INSERT INTO Student(Name)  
VALUES('Priya');  
INSERT INTO Student(Name)  
VALUES('Rahul');  
INSERT INTO Student(Name)  
VALUES('Kiran');
```

Output

ID	Name
1	Priya
2	Rahul
3	Kiran

Advantages

- Generates IDs automatically.
- Prevents duplicate IDs.
- Simplifies insertion.

Difference Between PRIMARY KEY and UNIQUE

Feature	PRIMARY KEY	UNIQUE
Duplicate Values	Not Allowed	Not Allowed
NULL Values	Not Allowed	One NULL value allowed
Number per Table	Only One	Multiple
Purpose	Identifies records uniquely	Prevents duplicates

Difference Between PRIMARY KEY and FOREIGN KEY

PRIMARY KEY	FOREIGN KEY
Uniquely identifies records	Creates relationships
Cannot contain NULL	Can contain NULL
One per table	Multiple allowed
Parent table column	Child table column

UNIT 7: Aggregate Functions

Aggregate functions are special SQL functions that perform calculations on multiple rows of a table and return a single value as the result.

These functions are commonly used to summarize data and generate reports.

Aggregate functions are widely used in:

- Data Analysis
- Reporting
- Business Intelligence
- Statistics
- Dashboard Applications

Why Do We Need Aggregate Functions?

Without aggregate functions:

- Calculations on large amounts of data become difficult.
- Finding totals, averages, and highest values requires manual effort.
- Report generation becomes time-consuming.

With aggregate functions:

- Data can be summarized easily.
- Reports can be generated quickly.
- Statistical analysis becomes simpler.

Aggregate Functions in SQL

SQL provides five major aggregate functions:

1. COUNT()
2. SUM()
3. AVG()
4. MAX()
5. MIN()

Sample Table

Consider the following Student table:

ID	Name	Marks
101	Priya	95
102	Rahul	88
103	Kiran	92
104	Anjali	85
105	Vijay	90

1. COUNT() Function :

The COUNT() function is used to count the number of rows in a table. It returns the total number of records that satisfy a specified condition.

Why Do We Need COUNT()?

COUNT() helps determine:

- Total number of students.

- Number of employees.
- Total orders placed.
- Number of products available.

Syntax

```
SELECT COUNT(column_name)
FROM table_name;
```

Example 1: Count Total Rows

```
SELECT COUNT(*)
FROM Student;
```

Output

5

Example 2: Count Specific Column

```
SELECT COUNT(Marks)
FROM Student;
```

Output

5

COUNT with WHERE Clause

```
SELECT COUNT(*)
FROM Student
WHERE Marks > 90;
```

Output

2

2. SUM() Function :

The SUM() function is used to calculate the total sum of numeric values in a column.

Why Do We Need SUM()?

SUM() helps calculate:

- Total salary.
- Total sales.
- Total marks.
- Total profit.

Syntax

```
SELECT SUM(column_name)
FROM table_name;
```

Example

```
SELECT SUM(Marks)
FROM Student;
```

Output

450

SUM with WHERE Clause

```
SELECT SUM(Marks)
FROM Student
WHERE Marks > 90;
```

Output

187

3. AVG() Function :

AVG() calculates the average value of a numeric column.

It divides the total sum by the number of rows.

Why Do We Need AVG()?

AVG() helps find:

- Average marks.
- Average salary.
- Average sales.
- Average rating.

Syntax

```
SELECT AVG(column_name)
FROM table_name;
```

Example

```
SELECT AVG(Marks)
FROM Student;
```

Output

90

AVG with WHERE Clause

```
SELECT AVG(Marks)
FROM Student
WHERE Marks > 85;
```

Output

91.25

4. MAX() Function :

MAX() returns the highest value from a column.

Why Do We Need MAX()?

MAX() helps find:

- Highest marks.
- Maximum salary.
- Maximum sales.
- Highest temperature.

Syntax

```
SELECT MAX(column_name)
FROM table_name;
```


Example

```
SELECT MAX(Marks)
FROM Student;
```

Output

95

MAX with WHERE Clause

```
SELECT MAX(Marks)
FROM Student
WHERE Marks < 95;
```

Output

92

5. MIN() Function :

MIN() returns the smallest value in a column.

Why Do We Need MIN()?

MIN() helps determine:

- Lowest marks.
- Minimum salary.
- Lowest temperature.
- Cheapest product.

Syntax

```
SELECT MIN(column_name)
FROM table_name;
```

Example

```
SELECT MIN(Marks)
FROM Student;
```

Output

85

MIN with WHERE Clause

```
SELECT MIN(Marks)
FROM Student
WHERE Marks > 85;
```

Output

88

Using Multiple Aggregate Functions Together

Example

```
SELECT
COUNT(*),
SUM(Marks),
AVG(Marks),
MAX(Marks),
MIN(Marks)
FROM Student;
```

Output

COUNT	SUM	AVG	MAX	MIN
5	450	90	95	85

Aggregate Functions with WHERE Clause

```
SELECT  
COUNT(*),  
AVG(Marks)  
FROM Student  
WHERE Marks >= 90;
```

Output

COUNT	AVG
3	92.33

Aggregate Functions and NULL Values

Consider:

ID	Marks
1	95
2	88
3	NULL
4	90

COUNT(Marks)

```
SELECT COUNT(Marks)  
FROM Student;
```

Output:

3

COUNT ignores NULL values.

COUNT(*) :

```
SELECT COUNT(*)  
FROM Student;
```

Output:

4

COUNT(*) includes all rows.

AVG(Marks) :

```
SELECT AVG(Marks)  
FROM Student;
```

Output:

91

NULL values are ignored.

Difference Between COUNT(*) and COUNT(Column)

COUNT(*)	COUNT(Column)
Counts all rows	Counts non-null values
Includes NULL rows	Ignores NULL values
Faster	Slightly slower

UNIT 8: String Functions

String functions are built-in SQL functions used to manipulate, modify, and process character data (text values).

These functions help perform operations such as:

- Converting text to uppercase or lowercase.
- Finding the length of strings.
- Combining strings.
- Extracting part of a string.
- Replacing characters or words.
- Removing unwanted spaces.

String functions are widely used in:

- Search systems
- Report generation
- Data cleaning
- User management systems
- E-commerce applications

Why Do We Need String Functions?

Without string functions:

- Text formatting becomes difficult.
- Searching and filtering data becomes complicated.
- Data cleaning requires manual effort.

With string functions:

- Text can be manipulated easily.
- Reports become more readable.
- Data can be standardized and cleaned.

Common String Functions

1. UPPER()
2. LOWER()
3. LENGTH()
4. CONCAT()
5. SUBSTRING()
6. REPLACE()
7. TRIM()

Sample Table

Employee Table

ID	Name	City
101	Priya	Hyderabad
102	Rahul	Chennai
103	Kiran	Vijayawada

1. UPPER() Function :

The UPPER() function converts all characters of a string into uppercase letters.

Why Do We Need UPPER()?

Sometimes data needs to be displayed in a standard format.

Examples:

- Names in reports.
- Product codes.
- User IDs.

Syntax

```
SELECT UPPER(column_name)
FROM table_name;
```

Example

```
SELECT UPPER(Name)
FROM Employee;
```

Output

```
UPPER(Name)
PRIYA
RAHUL
KIRAN
```

Example with String

```
SELECT UPPER('python');
```

Output

```
PYTHON
```

Applications

- Converting usernames to uppercase.
- Formatting reports.
- Data standardization.

2. LOWER() Function :

The LOWER() function converts all characters into lowercase letters.

Why Do We Need LOWER()?

Different users may enter text in different cases.

LOWER() helps maintain consistency.

Syntax

```
SELECT LOWER(column_name)
FROM table_name;
```

Example

```
SELECT LOWER(Name)
FROM Employee;
```

Output

LOWER(Name)
priya
rahul
kiran

Example with String

```
SELECT LOWER('PYTHON');
```

Output

python

Applications

- Email processing.
- Usernames.
- Search systems.

3. LENGTH() Function :

The LENGTH() function returns the number of characters present in a string.

Spaces are also counted as characters.

Why Do We Need LENGTH()?

LENGTH() is useful for:

- Password validation.
- Character counting.
- Input verification.

Syntax

```
SELECT LENGTH(column_name)
FROM table_name;
```

Example

```
SELECT LENGTH('Python');
```

Output

6

Example

```
SELECT LENGTH(Name)
FROM Employee;
```

Output

```
Name Length
Priya 5
Rahul 5
Kiran 5
```

Applications

- Password length checking.
- Username validation.
- Text analysis.

4. CONCAT() Function :

The CONCAT() function combines two or more strings into a single string.

Why Do We Need CONCAT()?

Sometimes information is stored in separate columns.

Example:

- First Name
- Last Name

These can be combined into a full name.

Syntax

```
SELECT CONCAT(string1, string2);
```

Example

```
SELECT CONCAT('Python',' SQL');
```

Output

Python SQL

Table Example

FirstName	LastName
Priya	Sharma
Rahul	Kumar

Query:

```
SELECT CONCAT(FirstName,' ',LastName)
FROM Employee;
```

Output

Full Name

Priya Sharma

Rahul Kumar

Applications

- Full names.
- Address generation.
- Product descriptions.

5. SUBSTRING() Function :

The SUBSTRING() function extracts a part of a string.

Why Do We Need SUBSTRING()?

Useful when only a portion of text is required.

Examples:

- First three characters of a code.
- Area code of phone numbers.
- Initials.

Syntax

```
SELECT SUBSTRING(string, start_position, length);
```

Example

```
SELECT SUBSTRING('DATABASE',1,4);
```

Output

DATA

Explanation

Starting position = 1

Length = 4

Characters extracted:

D A T A

Example

```
SELECT SUBSTRING(Name,1,3)
FROM Employee;
```

Output

Result

Pri

Rah

Kir

Applications

- Product codes.
- Phone numbers.
- User IDs.

6. REPLACE() Function :

The REPLACE() function replaces a specified part of a string with another string.

Why Do We Need REPLACE()?

Useful for:

- Correcting spelling mistakes.
- Updating names.
- Modifying text.

Syntax

```
SELECT REPLACE(string, old_value, new_value);
```

Example

```
SELECT REPLACE('Python','Python','Java');
```

Output

Java

Example

```
SELECT REPLACE('Welcome to SQL','SQL','MySQL');
```

Output

Welcome to MySQL

Applications

- Updating company names.
- Correcting words.
- Data cleaning.

7. TRIM() Function :

The TRIM() function removes unwanted spaces from the beginning and end of a string.

Why Do We Need TRIM()?

Users often enter extra spaces accidentally.
TRIM() helps clean the data.

Syntax

```
SELECT TRIM(string);
```

Example

```
SELECT TRIM(' Python ');
```

Output

Python

Without TRIM

" Python "

After TRIM

"Python"

Applications

- Cleaning user input.
- Form validation.
- Removing unwanted spaces.

Using Multiple String Functions Together

Example

```
SELECT UPPER(TRIM(' python programming '));
```

Output

PYTHON PROGRAMMING

Explanation

1. TRIM() removes spaces.
python programming
2. UPPER() converts text into uppercase.
PYTHON PROGRAMMING

Summary of String Functions

Function	Purpose
UPPER()	Converts to uppercase
LOWER()	Converts to lowercase
LENGTH()	Returns number of characters
CONCAT()	Combines strings
SUBSTRING()	Extracts part of a string
REPLACE()	Replaces characters or words
TRIM()	Removes leading and trailing spaces

UNIT 9: Date Functions

Date functions are built-in SQL functions used to work with date and time values. These functions help retrieve the current date, current time, year, month, day, and calculate differences between dates.

Date functions are widely used in:

- Employee Management Systems
- Banking Applications
- Hospital Management Systems
- E-Commerce Websites
- Reporting and Analytics Systems

Why Do We Need Date Functions?

Without date functions:

- Working with dates becomes difficult.
- Calculating age or service period requires manual effort.
- Generating reports based on dates becomes complicated.

With date functions:

- Date calculations become easier.
- Reports can be generated quickly.
- Date-based filtering becomes efficient.

Common Date Functions

1. CURDATE()
2. NOW()
3. YEAR()
4. MONTH()
5. DAY()
6. DATEDIFF()

1. CURDATE() Function :

The CURDATE() function returns the current system date. It displays only the date and does not include the time.

Why Do We Need CURDATE()?

CURDATE() is useful when:

- Recording today's date.
- Generating reports.
- Inserting registration dates.
- Tracking transactions.

Syntax

```
SELECT CURDATE();
```

Example

```
SELECT CURDATE();
```

Output

2025-07-20 (Output depends on the current system date.)

Example with Table

```
INSERT INTO Employee  
VALUES(101,'Priya',CURDATE());
```

Output

```
ID  Name  Joining_Date  
101 Priya  2025-07-20
```

2. NOW() Function :

The NOW() function returns the current date and current time.

Why Do We Need NOW()?

Useful for:

- Recording timestamps.
- Logging activities.
- Tracking transactions.
- Monitoring systems.

Syntax

```
SELECT NOW();
```

Example

```
SELECT NOW();
```

Output

2025-07-20 14:35:45 (The output depends on the system date and time.)

Difference Between CURDATE() and NOW()

CURDATE()	NOW()
Returns only date	Returns date and time
Format: YYYY-MM-DD	Format: YYYY-MM-DD HH:MM:SS

3. YEAR() Function :

The YEAR() function extracts the year part from a date.

Why Do We Need YEAR()?

Useful for:

- Annual reports.
- Filtering records by year.
- Age calculations.

Syntax

```
SELECT YEAR(date);
```

Example

```
SELECT YEAR('2025-07-20');
```

Output

```
2025
```

Example with Table

Employee Table

ID	Name	Joining_Date
101	Priya	2021-05-10
102	Rahul	2023-03-18

Query:

```
SELECT YEAR(Joining_Date)
FROM Employee;
```

Output

```
YEAR(Joining_Date)
2021
2023
```

4. MONTH() Function :

The MONTH() function extracts the month from a date.

The result ranges from 1 to 12.

Why Do We Need MONTH()?

Useful for:

- Monthly reports.
- Sales analysis.
- Attendance systems.

Syntax

```
SELECT MONTH(date);
```

Example

```
SELECT MONTH('2025-07-20');
```

Output

```
7
```

Example with Table

```
SELECT MONTH(Joining_Date)
FROM Employee;
```

Output

```
MONTH(Joining_Date)
5
3
```

5. DAY() Function :

The DAY() function extracts the day from a date.

The result ranges from 1 to 31.

Why Do We Need DAY()?

Useful for:

- Daily reports.
- Scheduling systems.
- Tracking events.

Syntax

```
SELECT DAY(date);
```

Example

```
SELECT DAY('2025-07-20');
```

Output :20

Example with Table

```
SELECT DAY(Joining_Date)
FROM Employee;
```

Output

```
DAY(Joining_Date)
10
18
```

6. DATEDIFF() Function :

The DATEDIFF() function returns the number of days between two dates.

Why Do We Need DATEDIFF()?

Useful for calculating:

- Employee experience.
- Customer subscription periods.
- Loan durations.
- Delivery times.

Syntax

```
SELECT DATEDIFF(date1, date2);
```

Example

```
SELECT DATEDIFF('2025-07-20','2025-07-15');
```

Output

```
5
```

Explanation

2025-07-20 - 2025-07-15 = 5 days

Example with Table

Employee Table

ID	Name	Joining_Date
101	Priya	2021-05-10

Query

```
SELECT DATEDIFF(CURDATE(), Joining_Date)
FROM Employee;
```

Output

```
1532(Output varies according to the current date.)
```

Using Multiple Date Functions Together

Example

```
SELECT
CURDATE(),
YEAR(CURDATE()),
MONTH(CURDATE()),
DAY(CURDATE());
```

Output

CURDATE()	YEAR()	MONTH()	DAY()
2025-07-20	2025	7	20

Summary of Date Functions

Function	Purpose
CURDATE()	Returns current date
NOW()	Returns current date and time
YEAR()	Extracts year
MONTH()	Extracts month
DAY()	Extracts day
DATEDIFF()	Finds difference between dates

Difference Between CURDATE() and NOW()

CURDATE()	NOW()
Returns only date	Returns date and time
YYYY-MM-DD	YYYY-MM-DD HH:MM:SS
Used in reports	Used in timestamps

Advantages of Date Functions

- Simplify date calculations.
- Generate reports efficiently.
- Support date-based filtering.
- Useful for analytics.
- Improve database management.

UNIT 10: GROUP BY and HAVING

GROUP BY and HAVING are SQL clauses used to organize rows into groups and apply conditions on those groups. They are generally used together with aggregate functions such as COUNT(), SUM(), AVG(), MAX(), and MIN(). These clauses are widely used in:

- Report generation
- Data analysis
- Sales reports
- Banking applications
- Employee management systems
- Business intelligence

Why Do We Need GROUP BY and HAVING?

Without GROUP BY:

- Data cannot be summarized category-wise.
- Aggregate functions work on the entire table only.

Without HAVING:

- Conditions cannot be applied to grouped data.

With GROUP BY and HAVING:

- Similar records can be grouped.
- Reports become easier to generate.
- Aggregate functions become more useful.

Sample Employee Table

EmpID	Name	Department	Salary
101	Priya	IT	50000
102	Rahul	HR	45000
103	Kiran	IT	60000
104	Anjali	Finance	55000
105	Ravi	HR	48000

1. GROUP BY Clause :

The GROUP BY clause is used to arrange rows having the same values into groups.

It is mainly used with aggregate functions to perform calculations on each group separately.

Why Do We Need GROUP BY?

GROUP BY helps:

- Count employees department-wise.
- Find average salary department-wise.
- Calculate total sales city-wise.
- Generate category-wise reports.

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

Example 1: Count Employees Department-wise

```
SELECT Department, COUNT(*)
FROM Employee
GROUP BY Department;
```

Output

Department	COUNT(*)
IT	2
HR	2
Finance	1

Explanation :

Employees are grouped according to their department, and COUNT() calculates the number of employees in each department.

Example 2: Find Total Salary Department-wise

```
SELECT Department, SUM(Salary)
FROM Employee
GROUP BY Department;
```

Output

Department	SUM(Salary)
IT	110000
HR	93000
Finance	55000

Example 3: Find Average Salary Department-wise

```
SELECT Department, AVG(Salary)
FROM Employee
GROUP BY Department;
```

Output

Department	AVG(Salary)
IT	55000
HR	46500
Finance	55000

Multiple Columns in GROUP BY

GROUP BY can be used with multiple columns.

Example

Student Table

ID	Name	City	Course
101	Priya	Hyderabad	Python
102	Rahul	Chennai	Java
103	Kiran	Hyderabad	Python
104	Anjali	Chennai	Python

Query:

```
SELECT City, Course, COUNT(*)  
FROM Student  
GROUP BY City, Course;
```

Output

City	Course	COUNT(*)
Hyderabad	Python	2
Chennai	Java	1
Chennai	Python	1

GROUP BY with MAX()

Example

```
SELECT Department, MAX(Salary)  
FROM Employee  
GROUP BY Department;
```

Output

Department	MAX(Salary)
IT	60000
HR	48000
Finance	55000

GROUP BY with MIN()

```
SELECT Department, MIN(Salary)  
FROM Employee  
GROUP BY Department;
```

Output

Department	MIN(Salary)
IT	50000
HR	45000
Finance	55000

2. HAVING Clause :

The HAVING clause is used to apply conditions on groups created by the GROUP BY clause.

HAVING works after grouping is completed.

Why Do We Need HAVING?

HAVING is useful when:

- Filtering grouped records.
- Finding departments with more than two employees.
- Displaying categories whose total sales exceed a certain amount.

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Example 1: Departments Having More Than One Employee

```
SELECT Department, COUNT(*)
FROM Employee
GROUP BY Department
HAVING COUNT(*) > 1;
```

Output

Department	COUNT(*)
IT	2
HR	2

Explanation :

Finance department contains only one employee, so it is excluded.

Example 2: Departments with Average Salary Greater Than 50000

```
SELECT Department, AVG(Salary)
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

Output

Department	AVG(Salary)
IT	55000
Finance	55000

Example 3: Departments with Total Salary Above 100000

```
SELECT Department, SUM(Salary)
FROM Employee
GROUP BY Department
HAVING SUM(Salary) > 100000;
```

Output

Department	SUM(Salary)
IT	110000

Difference Between WHERE and HAVING

Definition of WHERE :

WHERE filters rows before grouping takes place.

Definition of HAVING :

HAVING filters groups after grouping takes place.

Example Table

Department	Salary
IT	50000
IT	60000
HR	45000
HR	48000

Using WHERE

```
SELECT *  
FROM Employee  
WHERE Salary > 45000;
```

Output

Department	Salary
IT	50000
IT	60000
HR	48000

Explanation

WHERE filters individual rows.

Using HAVING

```
SELECT Department, AVG(Salary)  
FROM Employee  
GROUP BY Department  
HAVING AVG(Salary) > 50000;
```

Output

Department	AVG(Salary)
IT	55000

Explanation

HAVING filters grouped results.

Difference Between WHERE and HAVING

WHERE	HAVING
Filters rows	Filters groups
Executed before GROUP BY	Executed after GROUP BY
Cannot use aggregate functions directly	Works with aggregate functions
Used with SELECT, UPDATE, DELETE	Used with GROUP BY

GROUP BY with WHERE and HAVING Together

Example

```
SELECT Department, AVG(Salary)  
FROM Employee  
WHERE Salary > 45000  
GROUP BY Department  
HAVING AVG(Salary) > 50000;
```

Explanation

Step 1:

WHERE removes employees whose salary is less than or equal to 45000.

Step 2:

GROUP BY creates department-wise groups.

Step 3:

HAVING displays only those departments whose average salary exceeds 50000.

Output

Department	AVG(Salary)
IT	55000
Finance	55000

Execution Order

SQL processes the query in the following order:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

Summary

Clause	Purpose
GROUP BY	Groups rows with similar values
HAVING	Filters grouped records
WHERE	Filters individual rows
COUNT()	Counts records
SUM()	Finds total
AVG()	Calculates average
MAX()	Finds maximum value
MIN()	Finds minimum value

Advantages of GROUP BY and HAVING

- Simplify report generation.
- Provide summarized information.
- Work efficiently with aggregate functions.
- Help in business analysis.
- Widely used in real-world applications.

UNIT 11: JOINS

A JOIN is used to combine rows from two or more tables based on a related column between them.

Joins are one of the most powerful features of SQL because data in relational databases is usually stored in multiple tables. Joins help retrieve related information from these tables and display it as a single result.

Joins are widely used in:

- Employee Management Systems
- Banking Systems
- Hospital Management Systems
- E-Commerce Applications
- Student Management Systems

Why Do We Need Joins?

Without Joins:

- Data must be stored in a single table.
- Data redundancy increases.
- Database management becomes difficult.
- Retrieving related information becomes complicated.

With Joins:

- Data can be stored in separate tables.
- Data redundancy is reduced.
- Related information can be retrieved easily.
- Database normalization becomes possible.

Sample Tables

Employee Table

EmpID	Name	DeptID
101	Priya	1
102	Rahul	2
103	Kiran	1
104	Anjali	3

Department Table

DeptID	Department
1	IT
2	HR
3	Finance
4	Marketing

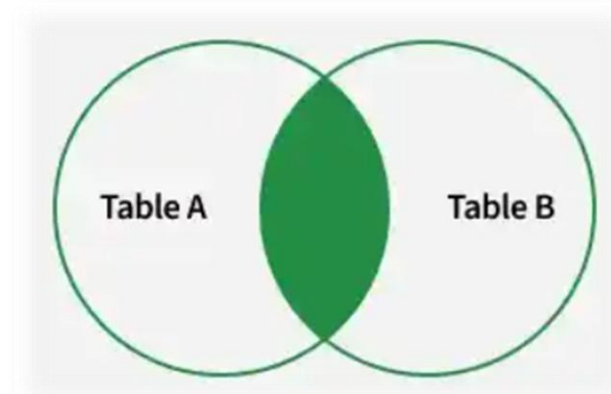
Types of Joins

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL OUTER JOIN
5. CROSS JOIN
6. SELF JOIN

1. INNER JOIN :

INNER JOIN returns only the rows that have matching values in both tables.

It ignores records that do not have matching values.



Syntax

```
SELECT columns  
FROM table1  
INNER JOIN table2  
ON table1.column = table2.column;
```

Example

```
SELECT Employee.Name, Department.Department  
FROM Employee  
INNER JOIN Department  
ON Employee.DeptID = Department.DeptID;
```

Output

Name	Department
Priya	IT
Rahul	HR
Kiran	IT
Anjali	Finance

Explanation :

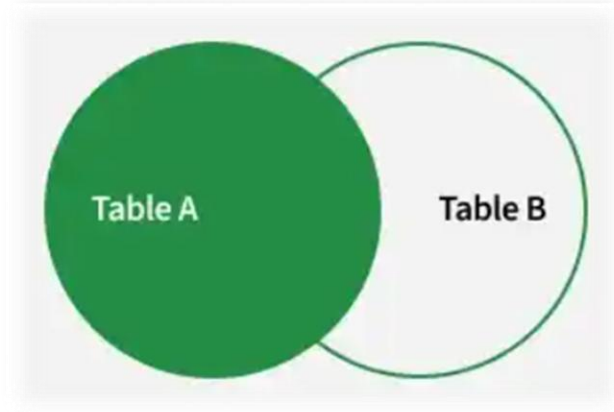
Each employee is matched with the corresponding department using DeptID.

2. LEFT JOIN :

LEFT JOIN returns:

- All rows from the left table.
- Matching rows from the right table.

If there is no match, NULL values are returned.



Syntax

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

Example

Suppose Employee table contains:

EmpID	Name	DeptID
101	Priya	1
102	Rahul	2
103	Kiran	5

Department Table:

DeptID	Department
1	IT
2	HR

Query

```
SELECT Employee.Name, Department.Department
FROM Employee
LEFT JOIN Department
ON Employee.DeptID = Department.DeptID;
```

Output

Name	Department
Priya	IT
Rahul	HR
Kiran	NULL

Explanation :

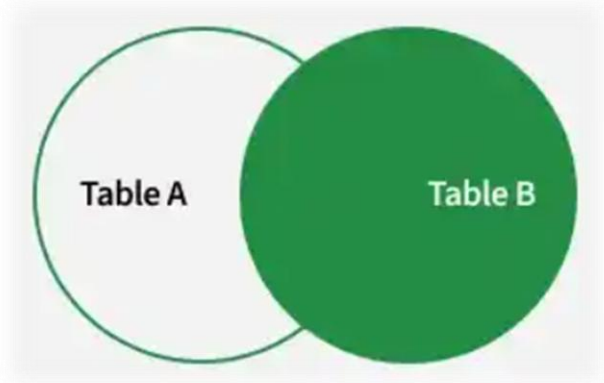
Kiran's DeptID does not exist in the Department table. Therefore, NULL is displayed.

3. RIGHT JOIN :

RIGHT JOIN returns:

- All rows from the right table.
- Matching rows from the left table.

If no match exists, NULL values are returned.

**Syntax**

```
SELECT columns  
FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column;
```

Example

Department Table

DeptID	Department
1	IT
2	HR
3	Finance

Employee Table

EmpID	Name	DeptID
101	Priya	1
102	Rahul	2

Query

```
SELECT Employee.Name, Department.Department  
FROM Employee  
RIGHT JOIN Department  
ON Employee.DeptID = Department.DeptID;
```

Output

Name	Department
Priya	IT
Rahul	HR
NULL	Finance

Explanation :

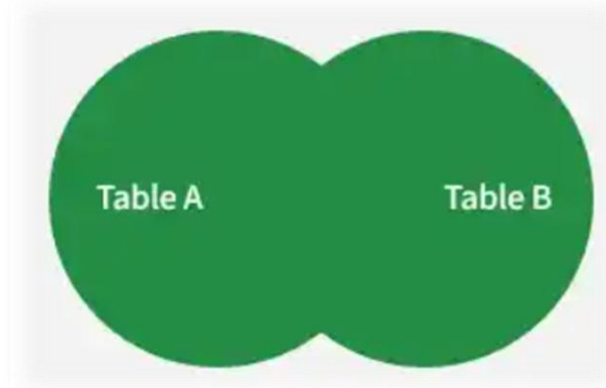
Finance department has no employee, so Name becomes NULL.

4. FULL OUTER JOIN :

FULL OUTER JOIN returns:

- All rows from the left table.
- All rows from the right table.

Matching rows are combined, and unmatched rows contain NULL values.

**Syntax**

```
SELECT columns  
FROM table1  
FULL OUTER JOIN table2  
ON table1.column = table2.column;
```

Example

Employee Table

EmpID	Name	DeptID
101	Priya	1
102	Rahul	2
103	Kiran	5

Department Table

DeptID	Department
1	IT
2	HR
3	Finance

Query

```
SELECT Employee.Name, Department.Department  
FROM Employee  
FULL OUTER JOIN Department  
ON Employee.DeptID = Department.DeptID;
```

Output

Name	Department
Priya	IT
Rahul	HR

Kiran	NULL
NULL	Finance

Explanation

Rows without matching values appear with NULL values.

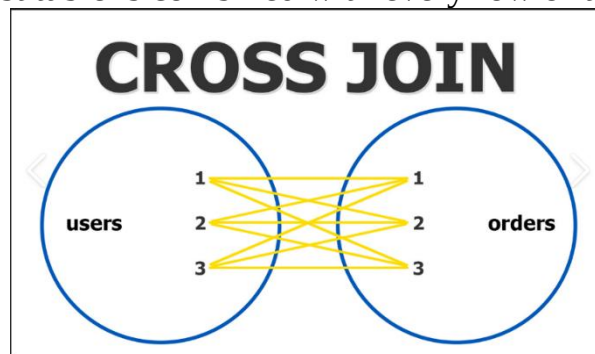
Applications

- Data analysis.
- Comparing two tables.
- Identifying missing records.

5. CROSS JOIN

CROSS JOIN returns the Cartesian product of two tables.

Every row of the first table is combined with every row of the second table.



Syntax

```
SELECT columns
FROM table1
CROSS JOIN table2;
```

Example

Colors Table

Color
Red
Blue

Sizes Table

Size
Small
Large

Query

```
SELECT *
FROM Colors
CROSS JOIN Sizes;
```

Output

Color	Size
Red	Small
Red	Large
Blue	Small
Blue	Large

Explanation

Number of rows produced:

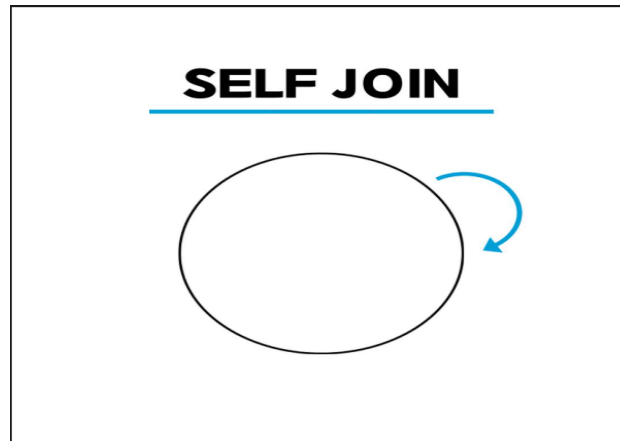
Rows in Table1 $\times$ Rows in Table2

$2 \times 2 = 4$ rows

6. SELF JOIN :

A SELF JOIN joins a table with itself.

It is used when rows within the same table are related.



Example Table

Employee Table

EmpID	Name	ManagerID
101	Priya	NULL
102	Rahul	101
103	Kiran	101
104	Anjali	102

Query

```
SELECT  
  E1.Name AS Employee,  
  E2.Name AS Manager  
FROM Employee E1  
LEFT JOIN Employee E2  
  ON E1.ManagerID = E2.EmpID;
```

Output

Employee	Manager
Priya	NULL
Rahul	Priya
Kiran	Priya
Anjali	Rahul

Explanation

The Employee table is joined with itself.

E1 represents employees.

E2 represents managers.

Difference Between Various Joins

Join	Returns
INNER JOIN	Matching rows from both tables
LEFT JOIN	All rows from left table + matching rows from right table
RIGHT JOIN	All rows from right table + matching rows from left table
FULL OUTER JOIN	All rows from both tables
CROSS JOIN	Cartesian product
SELF JOIN	Table joined with itself

Visual Representation

INNER JOIN

Table A $\cap$ Table B

Only common records.

LEFT JOIN

All A + Matching B

RIGHT JOIN

Matching A + All B

FULL OUTER JOIN

All A + All B

CROSS JOIN

A $\times$ B

SELF JOIN

Table A $\leftrightarrow$ Table A

Advantages of Joins

- Reduce data redundancy.
- Support database normalization.
- Retrieve related information efficiently.
- Improve data organization.
- Essential for relational databases.

Summary

Join Type	Description
INNER JOIN	Returns matching records
LEFT JOIN	Returns all left records
RIGHT JOIN	Returns all right records
FULL OUTER JOIN	Returns all records from both tables
CROSS JOIN	Produces Cartesian product
SELF JOIN	Joins a table with itself

UNIT 12: Set Operators

Set operators are special SQL operators used to combine the results of two or more SELECT statements into a single result set.

These operators are useful when data needs to be retrieved from multiple tables or when comparing datasets.

Set operators are widely used in:

- Reporting systems
- Data analysis
- Banking applications
- Employee management systems
- Business intelligence applications

Why Do We Need Set Operators?

Without set operators:

- Combining results from multiple queries becomes difficult.
- Data comparison requires complex logic.
- Duplicate handling becomes complicated.

With set operators:

- Results from multiple queries can be combined easily.
- Duplicate rows can be removed automatically.
- Data comparison becomes simpler.

Types of Set Operators

1. UNION
2. UNION ALL
3. INTERSECT
4. MINUS (EXCEPT)

Rules for Using Set Operators

Before using set operators, the following conditions must be satisfied:

1. Number of Columns Must Be Same

Correct:

```
SELECT Name, City
FROM Student1
UNION
SELECT Name, City
FROM Student2;
```

2. Data Types Must Be Compatible

Correct:

```
VARCHAR ↔ VARCHAR
INTEGER ↔ INTEGER
```

3. Column Names Need Not Be Same

Only the number and data types should match.

Sample Tables

Student2024

ID	Name
101	Priya
102	Rahul
103	Kiran

Student2025

ID	Name
103	Kiran
104	Anjali
105	Vijay

1. UNION Operator :

The UNION operator combines the results of two or more SELECT statements and automatically removes duplicate rows.

Syntax

```
SELECT column_name  
FROM table1  
UNION  
SELECT column_name  
FROM table2;
```

Example

```
SELECT Name  
FROM Student2024  
UNION  
SELECT Name  
FROM Student2025;
```

Output

Name
Priya
Rahul
Kiran
Anjali
Vijay

Explanation

Kiran appears in both tables.

UNION removes duplicate rows automatically.

UNION with Multiple Columns

```
SELECT ID, Name
FROM Student2024
UNION
SELECT ID, Name
FROM Student2025;
```

Output

ID	Name
101	Priya
102	Rahul
103	Kiran
104	Anjali
105	Vijay

2. UNION ALL Operator :

UNION ALL combines the results of multiple SELECT statements but does not remove duplicate rows.

All records are returned exactly as they exist.

Syntax

```
SELECT column_name
FROM table1
UNION ALL
SELECT column_name
FROM table2;
```

Example

```
SELECT Name
FROM Student2024
UNION ALL
SELECT Name
FROM Student2025;
```

Output

Name
Priya
Rahul
Kiran
Kiran
Anjali
Vijay

Explanation

Since Kiran exists in both tables, it appears twice.

Difference Between UNION and UNION ALL

UNION	UNION ALL
Removes duplicates	Keeps duplicates
Slower	Faster
Performs sorting	No sorting
Returns unique rows	Returns all rows

3. INTERSECT Operator

INTERSECT returns only the common rows that exist in both result sets. It finds the intersection of two tables.

Syntax

```
SELECT column_name
FROM table1
INTERSECT
SELECT column_name
FROM table2;
```

Example

```
SELECT Name
FROM Student2024
INTERSECT
SELECT Name
FROM Student2025;
```

Output

```
Name
Kiran
```

Explanation

Kiran exists in both tables.
Therefore, only Kiran is returned.

4. MINUS Operator :

MINUS returns rows that exist in the first query but not in the second query.

Syntax

```
SELECT column_name
FROM table1
MINUS
SELECT column_name
FROM table2;
```

Example

```
SELECT Name
FROM Student2024
MINUS
SELECT Name
FROM Student2025;
```

Output

Name

Priya

Rahul

Explanation

Kiran is present in both tables, so it is removed.

Only records existing exclusively in Student2024 are returned.

EXCEPT Operator

Some databases use EXCEPT instead of MINUS.

Example:

```
SELECT Name
FROM Student2024
EXCEPT
SELECT Name
FROM Student2025;
```

Output

Name

Priya

Rahul

Visual Representation

Suppose:

Table A

$A = \{1,2,3,4\}$

Table B

$B = \{3,4,5,6\}$

UNION

$A \cup B$

$\{1,2,3,4,5,6\}$

UNION ALL

$\{1,2,3,4,3,4,5,6\}$

INTERSECT

$A \cap B$

$\{3,4\}$

MINUS

$A - B$

$\{1,2\}$

Set Operators with WHERE Clause

Example

```
SELECT Name
FROM Student2024
WHERE ID > 101
UNION
```



```
SELECT Name
FROM Student2025
WHERE ID < 105;
```

Output

```
Name
Rahul
Kiran
Anjali
```

Set Operators with ORDER BY

```
SELECT Name
FROM Student2024
UNION
SELECT Name
FROM Student2025
ORDER BY Name;
```

Output

```
Name
Anjali
Kiran
Priya
Rahul
Vijay
```

Comparison of Set Operators

Operator	Duplicate Rows	Purpose
UNION	Removed	Combine unique records
UNION ALL	Preserved	Combine all records
INTERSECT	Removed	Common records
MINUS	Removed	Difference between tables

Advantages of Set Operators

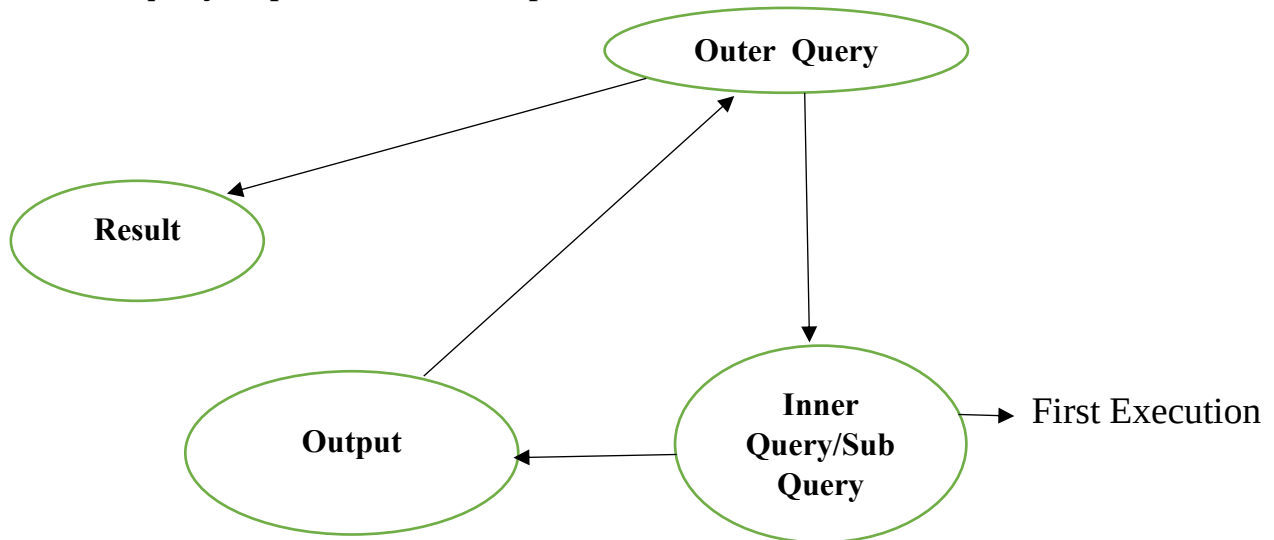
- Combine results from multiple queries.
- Simplify data analysis.
- Eliminate duplicates when necessary.
- Improve query readability.
- Support reporting and analytics.

Summary

Operator	Description
UNION	Combines results and removes duplicates
UNION ALL	Combines results and keeps duplicates
INTERSECT	Returns common records
MINUS	Returns records existing only in first query

UNIT 13: Subqueries

A Subquery is a query written inside another SQL query. It is also known as an inner query or nested query. The outer query uses the result produced by the inner query to perform further operations.



Subqueries help retrieve data based on the result of another query and are widely used in:

- Employee Management Systems
- Banking Systems
- E-Commerce Applications
- Hospital Management Systems
- Reporting and Analytics

Why Do We Need Subqueries?

Without subqueries:

- Complex problems require multiple separate queries.
- Data retrieval becomes difficult.
- Query execution becomes less efficient.

With subqueries:

- Complex queries become easier.
- Results of one query can be used by another query.
- Data analysis becomes simpler.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name operator
(
    SELECT column_name
    FROM table_name
);
```

Sample Employee Table

EmpID	Name	Salary	Department
101	Priya	50000	IT
102	Rahul	45000	HR
103	Kiran	60000	IT
104	Anjali	55000	Finance
105	Vijay	48000	HR

Types of Subqueries

1. Single-row Subquery
2. Multiple-row Subquery
3. Correlated Subquery
4. Nested Subquery

1. Single-row Subquery

A Single-row Subquery returns only one row and one value.

These subqueries are commonly used with:

- =
- >=
- <
- <=
- <>= (Not Equal to)

comparison operators.

Example 1

Find employees whose salary is greater than the average salary.

Query

```
SELECT Name, Salary
FROM Employee
WHERE Salary >
(
    SELECT AVG(Salary)
    FROM Employee
);
```

Step 1: Inner Query

```
SELECT AVG(Salary)
FROM Employee;
```

Output

51600

Step 2: Outer Query

```
SELECT Name, Salary
FROM Employee
WHERE Salary > 51600;
```

Output

Name	Salary
Kiran	60000
Anjali	55000

Example 2

Find the employee having the highest salary.

```
SELECT Name, Salary
```

```
FROM Employee
```

```
WHERE Salary =
```

```
(  
    SELECT MAX(Salary)  
    FROM Employee  
);
```

Output

Name	Salary
Kiran	60000

2. Multiple-row Subquery :

A Multiple-row Subquery returns more than one row.

These subqueries are generally used with:

- IN
- ANY
- ALL

operators.

Example 1 Using IN

Suppose we have another table.

Manager Table

Department
IT
Finance

Find employees belonging to departments present in the Manager table.

```
SELECT Name, Department
```

```
FROM Employee
```

```
WHERE Department IN
```

```
(  
    SELECT Department  
    FROM Manager  
);
```

Output

Name	Department
Priya	IT
Kiran	IT
Anjali	Finance

Example Using ANY

```
SELECT Name, Salary
FROM Employee
WHERE Salary >
ANY
(
    SELECT Salary
    FROM Employee
    WHERE Department='HR'
);
```

Inner Query Output

45000
48000

Output

Name	Salary
Priya	50000
Kiran	60000
Anjali	55000

Example Using ALL

```
SELECT Name, Salary
FROM Employee
WHERE Salary >
ALL
(
    SELECT Salary
    FROM Employee
    WHERE Department='HR'
);
```

Output

Name	Salary
Priya	50000
Kiran	60000
Anjali	55000

3. Correlated Subquery :

A Correlated Subquery is a subquery that depends on the outer query. The inner query executes once for each row processed by the outer query.

Syntax

```
SELECT column_name
FROM table1 t1
WHERE condition
(
    SELECT column_name
    FROM table2 t2
```

WHERE t1.column = t2.column

);

Example

Find employees whose salary is greater than the average salary of their department.

Employee Table

Name	Salary	Department
Priya	50000	IT
Kiran	60000	IT
Rahul	45000	HR
Vijay	48000	HR
Anjali	55000	Finance

Query

```
SELECT Name, Salary, Department
FROM Employee E1
WHERE Salary >
(
    SELECT AVG(Salary)
    FROM Employee E2
    WHERE E1.Department = E2.Department
);
```

Output

Name	Salary	Department
Kiran	60000	IT
Vijay	48000	HR

Explanation

Average salary in IT:

55000

Kiran's salary:

60000

Therefore, Kiran is selected.

Average salary in HR:

46500

Vijay earns 48000, which is greater than the department average.

4. Nested Subquery :

A Nested Subquery is a query containing another subquery inside it. In other words, there are multiple levels of subqueries.

Example

Suppose we have:

Department Table

DeptID	Department
1	IT
2	HR

Employee Table

Name	DeptID
Priya	1
Rahul	2
Kiran	1

Find employees working in the IT department.

Query

```
SELECT Name
FROM Employee
WHERE DeptID =
(
    SELECT DeptID
    FROM Department
    WHERE Department =
    (
        SELECT 'IT'
    )
);
```

Output

```
Name
Priya
Kiran
```

Explanation

Level 1

```
SELECT 'IT'
```

returns

```
IT
```

Level 2

```
SELECT DeptID
FROM Department
WHERE Department='IT';
```

returns

```
1
```

Level 3

```
SELECT Name
FROM Employee
WHERE DeptID=1;
```

returns

```
Priya
Kiran
```

Subquery with EXISTS

EXISTS checks whether the subquery returns any rows.

If rows exist, EXISTS returns TRUE.

Example

```
SELECT Name
FROM Employee E
WHERE EXISTS
(
    SELECT *
    FROM Department D
    WHERE E.Department = D.Department
);
```

Output

All employees whose departments exist in the Department table.

Subquery with NOT EXISTS

```
SELECT Name
FROM Employee E
WHERE NOT EXISTS
(
    SELECT *
    FROM Department D
    WHERE E.Department = D.Department
);
```

Output

Employees whose departments do not exist in the Department table.

Subqueries in SELECT Statement

```
SELECT Name,
(
    SELECT AVG(Salary)
    FROM Employee
) AS AverageSalary
FROM Employee;
```

Output

Name	AverageSalary
Priya	51600
Rahul	51600

Kiran	51600
Anjali	51600
Vijay	51600

Subqueries in FROM Clause

```
SELECT AVG(TotalSalary)
FROM
(
    SELECT SUM(Salary) AS TotalSalary
    FROM Employee
    GROUP BY Department
) AS Temp;
```

Output

Average of department salaries.

Difference Between Single-row and Multiple-row Subqueries

Single-row Subquery	Multiple-row Subquery
Returns one row	Returns multiple rows
Uses =, >, <	Uses IN, ANY, ALL
Simpler	More flexible

Difference Between Normal and Correlated Subquery

Normal Subquery	Correlated Subquery
Executes once	Executes for every row
Independent	Depends on outer query
Faster	Comparatively slower

Advantages of Subqueries

- Simplify complex queries.
- Improve readability.
- Support dynamic data retrieval.
- Reduce the need for multiple queries.
- Useful in reporting and analytics.

Summary

Type	Description
Single-row Subquery	Returns one row
Multiple-row Subquery	Returns multiple rows
Correlated Subquery	Depends on outer query
Nested Subquery	Contains multiple levels of queries

UNIT 14: Views

A View is a virtual table in SQL that is created from one or more existing tables. Unlike regular tables, a view does not store data physically. Instead, it stores the SQL query used to generate the data.

Whenever a view is accessed, SQL executes the underlying query and displays the latest data from the original table(s).

Views are mainly used to:

- Simplify complex queries.
- Improve data security.
- Restrict access to sensitive information.
- Present data in a customized format.

Why Do We Need Views?

Without Views:

- Complex queries must be written repeatedly.
- Users may access confidential data.
- Query maintenance becomes difficult.

With Views:

- Complex queries become simpler.
- Data security improves.
- Queries become reusable.
- Different users can see different portions of the same data.

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Sample Employee Table

EmpID	Name	Salary	Department
101	Priya	50000	IT
102	Rahul	45000	HR
103	Kiran	60000	IT
104	Anjali	55000	Finance
105	Vijay	48000	HR

Types of Views

1. Simple View
2. Complex View

1.Simple View :

A Simple View is created from a single table and does not contain aggregate functions or GROUP BY clauses.

Example

```
CREATE VIEW Employee_View AS
SELECT EmpID, Name, Department
FROM Employee;
```

Accessing the View

```
SELECT *
FROM Employee_View;
```

Output

EmpID	Name	Department
101	Priya	IT
102	Rahul	HR
103	Kiran	IT
104	Anjali	Finance
105	Vijay	HR

2.Complex View :

A Complex View is created using:

- Multiple tables
- Aggregate functions
- GROUP BY clause
- Joins

Example

Suppose we have:

Department Table

DeptID	Department
1	IT
2	HR
3	Finance

Employee Table

EmpID	Name	DeptID
101	Priya	1
102	Rahul	2
103	Kiran	1
104	Anjali	3

Creating a Complex View

```
CREATE VIEW EmployeeDepartment AS
SELECT E.Name, D.Department
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;
```

Accessing the View

```
SELECT *
FROM EmployeeDepartment;
```

Output

```
Name Department
Priya IT
Rahul HR
Kiran IT
Anjali Finance
```

CREATE VIEW :

CREATE VIEW is used to create a virtual table based on one or more tables.

Syntax

```
CREATE VIEW view_name AS
SELECT columns
FROM table_name;
```

Example

```
CREATE VIEW IT_Employees AS
SELECT Name, Salary
FROM Employee
WHERE Department='IT';
```

Accessing the View

```
SELECT *
FROM IT_Employees;
```

Output

```
Name Salary
Priya 50000
Kiran 60000
```

Explanation

Only employees belonging to the IT department are displayed.

Advantages of CREATE VIEW

- Reduces query complexity.
- Increases reusability.
- Enhances security.
- Simplifies reporting.

UPDATE VIEW

Data inside a view can be updated if the view is based on a single table and does not contain:

- Aggregate functions
- GROUP BY clause
- DISTINCT
- Joins

Changes made to the view affect the original table.

Example

Suppose the view is:

```
CREATE VIEW Employee_View AS
SELECT EmpID, Name, Salary
FROM Employee;
```

Updating Data Through View

```
UPDATE Employee_View
SET Salary = 55000
WHERE EmpID = 101;
```

Employee Table After Update

EmpID	Name	Salary
101	Priya	55000
102	Rahul	45000
103	Kiran	60000

Explanation

Updating the view automatically updates the original Employee table.

INSERT Through View

```
INSERT INTO Employee_View
VALUES (106, 'Arjun', 47000);
```

Output

A new record is inserted into the Employee table.

DELETE Through View

```
DELETE FROM Employee_View
WHERE EmpID = 102;
```

Result

Employee 102 is deleted from the Employee table.

DROP VIEW

DROP VIEW removes a view permanently from the database.

The original tables remain unaffected.

Syntax

```
DROP VIEW view_name;
```

Example

```
DROP VIEW Employee_View;
```

Explanation

The view is deleted, but the Employee table still exists.

Example

Suppose:

```
CREATE VIEW HR_Employees AS
SELECT Name
FROM Employee
WHERE Department='HR';
```

Before Dropping

```
SELECT *  
FROM HR_Employees;
```

Output:

```
Name  
Rahul  
Vijay
```

Drop the View

```
DROP VIEW HR_Employees;
```

Access Again

```
SELECT *  
FROM HR_Employees;
```

Output:

```
ERROR: View does not exist
```

Updating a View Definition

Some databases support:

```
CREATE OR REPLACE VIEW view_name AS  
SELECT columns  
FROM table_name;
```

Example

```
CREATE OR REPLACE VIEW Employee_View AS  
SELECT Name, Department  
FROM Employee;
```

Benefits of Views

1. Simplifies Complex Queries

Instead of writing:

```
SELECT E.Name, D.Department  
FROM Employee E  
INNER JOIN Department D  
ON E.DeptID = D.DeptID;
```

Use:

```
SELECT *  
FROM EmployeeDepartment;
```

2. Improves Security

Sensitive columns can be hidden.

Original Table:

Name	Salary	Department
Priya	50000	IT

View:

```
CREATE VIEW Public_View AS  
SELECT Name, Department  
FROM Employee;
```

Users cannot see salary information.

3. Provides Data Independence

Changes in table structure can often be hidden from users.

4. Reusability

Frequently used queries can be stored as views.

Difference Between Table and View

Table	View
Stores data physically	Does not store data physically
Occupies storage space	Requires little storage
Can exist independently	Depends on tables
Faster access	Slightly slower
Stores records	Stores query

Difference Between Simple and Complex Views

Simple View	Complex View
Based on one table	Based on multiple tables
Supports INSERT, UPDATE, DELETE	Limited support
No aggregate functions	Contains aggregate functions
Easy to modify	Difficult to modify

Advantages of Views

- Simplify complex queries.
- Improve security.
- Increase code reusability.
- Provide abstraction.
- Reduce query duplication.
- Improve maintainability.

Disadvantages of Views

- Slightly slower than tables.
- Complex views cannot always be updated.
- Depend on underlying tables.
- Dropping a table may affect dependent views.

Summary

Command	Purpose
CREATE VIEW	Creates a new view
UPDATE VIEW	Modifies data through the view
DROP VIEW	Removes the view

UNIT 15: Indexes

An Index is a database object used to improve the speed of data retrieval operations on a table. It works similarly to the index of a book. Instead of searching every page one by one, we can use the index to quickly locate the required information.

Indexes help SQL find rows faster without scanning the entire table.

Indexes are widely used in:

- Banking systems
- Employee management systems
- E-commerce applications
- Hospital management systems
- Reporting and analytics systems

Why Do We Need Indexes?

Without Indexes:

- SQL scans the entire table.
- Query execution becomes slower.
- Performance decreases for large tables.

With Indexes:

- Data retrieval becomes faster.
- Query performance improves.
- Searching records becomes efficient.
- Database applications become more responsive.

Example of a Book Index

Suppose you want to find the topic "SQL Joins" in a 500-page book.

Without an index:

- You must search page by page.

With an index:

- You can directly go to the page number mentioned in the index.

Similarly, SQL indexes help locate rows quickly.

Types of Indexes

1. CREATE INDEX (Normal Index)
2. UNIQUE INDEX
3. DROP INDEX

Some databases also support:

- Clustered Index
- Non-Clustered Index
- Composite Index
- Full Text Index

Sample Employee Table

EmpID	Name	Department	Salary
101	Priya	IT	50000
102	Rahul	HR	45000
103	Kiran	IT	60000
104	Anjali	Finance	55000
105	Vijay	HR	48000

1.CREATE INDEX :

CREATE INDEX is used to create an index on one or more columns of a table to improve query performance.

The values in the indexed column are arranged in a structure that allows SQL to locate records quickly.

Syntax

```
CREATE INDEX index_name  
ON table_name(column_name);
```

Example

```
CREATE INDEX idx_name  
ON Employee(Name);
```

Explanation

An index named:

idx_name

is created on the Name column.

Searching for employee names becomes faster.

Query Example

```
SELECT *  
FROM Employee  
WHERE Name='Priya';
```

Output

```
EmpID Name Department Salary  
101    Priya  IT          50000
```

Creating Index on Multiple Columns :

An index can be created on more than one column.

This is called a Composite Index.

Syntax

```
CREATE INDEX index_name  
ON table_name(column1, column2);
```

Example

```
CREATE INDEX idx_emp  
ON Employee(Name, Department);
```

Explanation

The index is created using:

- Name

- Department

Queries involving these columns execute faster.

2.UNIQUE INDEX :

A UNIQUE INDEX ensures that duplicate values are not allowed in the indexed column.

It improves performance and maintains data integrity.

Syntax

```
CREATE UNIQUE INDEX index_name
ON table_name(column_name);
```

Example

```
CREATE UNIQUE INDEX idx_email
ON Employee(Email);
```

Suppose:

```
Email
priya@gmail.com
rahul@gmail.com
```

Trying to insert:

```
priya@gmail.com
again will generate an error because duplicates are not allowed.
```

Example

```
INSERT INTO Employee
VALUES
(106,'Arjun','IT',47000,'priya@gmail.com');
```

Output

```
ERROR: Duplicate value violates UNIQUE INDEX
```

3.DROP INDEX :

DROP INDEX removes an existing index from the database.

Removing an index does not affect the data inside the table.

Syntax

MySQL

```
DROP INDEX index_name
ON table_name;
```

SQL Server

```
DROP INDEX table_name.index_name;
```

Example

```
DROP INDEX idx_name
ON Employee;
```

Explanation

The index:

```
idx_name
```

is removed, but all employee records remain unchanged.

Example

Before dropping:

```
CREATE INDEX idx_name  
ON Employee(Name);
```

After dropping:

```
DROP INDEX idx_name  
ON Employee;
```

The Employee table still contains:

EmpID	Name
101	Priya
102	Rahul
103	Kiran

4.Clustered Index :

A Clustered Index determines the physical order in which data is stored in a table.

Only one clustered index can exist per table.

Example

Suppose data is arranged based on EmpID.

101
102
103
104
105

The rows themselves are physically stored in this order.

Applications

- Primary Key columns.
- Frequently sorted columns.

5.Non-Clustered Index :

A Non-Clustered Index stores a separate structure that points to the actual data rows.

Multiple non-clustered indexes can exist in a table.

Example

Suppose an index is created on:

Name

The index stores:

Priya → Row 1
Rahul → Row 2
Kiran → Row 3

6.Composite Index :

A Composite Index uses multiple columns.

Example

```
CREATE INDEX idx_emp  
ON Employee(Name, Department);
```

Useful query:

```
SELECT *  
FROM Employee  
WHERE Name='Priya'  
AND Department='IT';
```

7.Full Text Index :

Full Text Indexes are used for searching text data efficiently.

Example

Product Table

ProductDescription
Laptop with 16GB RAM
Gaming Keyboard
Wireless Mouse

Search:

```
SELECT *  
FROM Product  
WHERE MATCH(ProductDescription)  
AGAINST('Laptop');
```

Output

Laptop with 16GB RAM

Advantages of Indexes**1.Faster Query Execution**

Indexes reduce searching time.

2.Better Performance

Large databases perform efficiently.

3.Reduced Disk Access

Fewer rows need to be scanned.

4.Faster Sorting

ORDER BY operations become quicker.

5.Improved JOIN Performance

Joins execute faster when indexed columns are used.

Disadvantages of Indexes**1.Additional Storage Space**

Indexes consume extra memory.

2.Slower INSERT Operations

Whenever data is inserted, indexes must also be updated.

3.Slower UPDATE Operations

Updating indexed columns requires index maintenance.

4.Slower DELETE Operations

Deleting rows also affects indexes.

Difference Between Index and Primary Key

Primary Key	Index
Uniquely identifies rows	Improves performance
Duplicates not allowed	Duplicates allowed (except UNIQUE INDEX)
One primary key per table	Multiple indexes possible
Mandatory for identification	Optional

Summary

Command	Purpose
CREATE INDEX	Creates an index
CREATE UNIQUE INDEX	Creates index with unique values
DROP INDEX	Removes an index

UNIT 16: TCL Commands

Transaction Control Language (TCL) commands are used to manage transactions in a database. A transaction is a group of SQL statements that are executed as a single unit of work.

TCL commands help maintain data consistency and ensure that changes made to the database are either completely saved or completely undone.

TCL commands are widely used in:

- Banking Systems
- Online Payment Systems
- E-Commerce Applications
- Airline Reservation Systems
- Inventory Management Systems

What is a Transaction?

A transaction is a sequence of one or more SQL statements executed together as a single logical unit.

A transaction should satisfy the ACID properties to maintain database integrity.

Example:

```
UPDATE Account
SET Balance = Balance - 5000
WHERE AccountNo = 101;
UPDATE Account
SET Balance = Balance + 5000
WHERE AccountNo = 102;
```

Both statements together form one transaction.

Why Do We Need TCL Commands?

Without TCL Commands:

- Partial updates may occur.
- Data inconsistency can happen.
- Errors may corrupt the database.

With TCL Commands:

- Data remains consistent.
- Transactions can be committed or rolled back.
- Errors can be handled safely.
- Database reliability improves.

Types of TCL Commands

1. COMMIT
2. ROLLBACK
3. SAVEPOINT

1. COMMIT :

COMMIT is used to permanently save all changes made during the current transaction.

Once a transaction is committed, the changes become permanent and cannot be undone using ROLLBACK.

Syntax

COMMIT;

Example

Employee Table

EmpID Name Salary

101 Priya 50000

102 Rahul 45000

Update Salary

UPDATE Employee

SET Salary = 55000

WHERE EmpID = 101;

After updating:

EmpID	Name	Salary
101	Priya	55000
102	Rahul	45000

Save the changes:

COMMIT;

Explanation

The updated salary is permanently stored in the database.

2. ROLLBACK :

ROLLBACK is used to undo changes made during the current transaction. It restores the database to its previous state before the transaction began.

Syntax

ROLLBACK;

Example

Original Table

AccountNo	Balance
101	20000
102	15000

Deduct money:

UPDATE Account

SET Balance = Balance - 5000

WHERE AccountNo = 101;

Table becomes:

AccountNo	Balance
101	15000
102	15000

Suppose an error occurs.

Execute:

ROLLBACK;

Output

AccountNo	Balance
101	20000
102	15000

Explanation

ROLLBACK cancels the transaction and restores the original values.

Example of COMMIT and ROLLBACK

```
UPDATE Employee  
SET Salary = 60000  
WHERE EmpID = 101;
```

Table:

EmpID	Salary
101	60000

Case 1

```
COMMIT;
```

Result:

Salary becomes permanent.

Case 2

```
ROLLBACK;
```

Result:

Salary returns to its previous value.

3. SAVEPOINT :

SAVEPOINT creates a checkpoint within a transaction.

It allows partial rollback instead of rolling back the entire transaction.

Syntax

```
SAVEPOINT savepoint_name;
```

Example

Employee Table

EmpID	Salary
101	50000
102	45000

Transaction:

```
UPDATE Employee  
SET Salary = 55000  
WHERE EmpID = 101;
```

Create Savepoint:

```
SAVEPOINT A;
```

Update another employee:

```
UPDATE Employee  
SET Salary = 48000  
WHERE EmpID = 102;
```


Current Table

EmpID Salary

101 55000

102 48000

Rollback to Savepoint:

ROLLBACK TO A;

Output

EmpID Salary

101 55000

102 45000

Explanation

Only changes after SAVEPOINT A are undone.
The first update remains unchanged.

Complete Example

UPDATE Account

SET Balance = Balance - 1000

WHERE AccountNo = 101;

SAVEPOINT SP1;

UPDATE Account

SET Balance = Balance + 1000

WHERE AccountNo = 102;

ROLLBACK TO SP1;

COMMIT;

Explanation

Step 1:

Account 101 balance decreases.

Step 2:

SAVEPOINT SP1 is created.

Step 3:

Account 102 balance increases.

Step 4:

ROLLBACK TO SP1 removes only Step 3.

Step 5:

COMMIT permanently saves Step 1.

ACID Properties

Transactions should satisfy ACID properties.

1. Atomicity :

Either all operations are completed successfully, or none are performed.

Example:

Money transfer:

Account A → Debit ₹5000

Account B → Credit ₹5000

Both operations should happen together.

2. Consistency :

A transaction should move the database from one valid state to another valid state.

Example:

Total money before transfer:

50000

Total money after transfer:

50000

Consistency is maintained.

3. Isolation :

Transactions should not interfere with each other.

Multiple users can perform operations independently.

4. Durability :

Once COMMIT is executed, changes are permanently stored even if the system crashes.

Difference Between COMMIT and ROLLBACK

COMMIT	ROLLBACK
Saves changes permanently	Cancels changes
Cannot be undone	Restores previous state
Ends transaction	Returns to original data
Used after successful execution	Used after errors

Difference Between ROLLBACK and SAVEPOINT

ROLLBACK	SAVEPOINT
Undoes complete transaction	Creates checkpoints
Returns to initial state	Returns to intermediate state
Simple recovery	Partial recovery

Advantages of TCL Commands

- Maintain data consistency.
- Support error recovery.
- Ensure reliable transactions.
- Improve database integrity.
- Provide partial rollback using SAVEPOINT.

Summary

Command	Purpose
COMMIT	Permanently saves changes
ROLLBACK	Cancels changes
SAVEPOINT	Creates checkpoints within transactions

UNIT 17: DCL Commands

Data Control Language (DCL) commands are used to control access to the database. These commands help database administrators manage permissions and privileges for users.

DCL commands determine:

- Who can access the database.
- What operations a user can perform.
- Which tables or objects a user can access.

DCL commands are mainly used to provide security and maintain controlled access to database resources.

Why Do We Need DCL Commands?

Without DCL Commands:

- Every user could access all tables.
- Sensitive information might be exposed.
- Database security would be weak.
- Unauthorized users could modify or delete data.

With DCL Commands:

- Access can be restricted.
- Security is improved.
- Permissions can be assigned based on roles.
- Sensitive data remains protected.

Types of DCL Commands

1. GRANT
2. REVOKE

Sample Employee Table

EmpID	Name	Salary
101	Priya	50000
102	Rahul	45000
103	Kiran	60000

Suppose there are two users:

- Admin
- User1

The administrator controls which operations User1 can perform.

1. GRANT Command :

The GRANT command is used to provide privileges or permissions to users on database objects such as tables, views, and procedures.

It specifies what actions a user is allowed to perform.

Syntax

```
GRANT privilege_name
ON table_name
TO user_name;
```

Example 1: Grant SELECT Permission

```
GRANT SELECT  
ON Employee  
TO User1;
```

Explanation

User1 is allowed to:

```
SELECT * FROM Employee;
```

But cannot:

```
INSERT  
UPDATE  
DELETE
```

Example 2: Grant Multiple Permissions

```
GRANT SELECT, INSERT  
ON Employee  
TO User1;
```

User1 Can Perform

```
SELECT * FROM Employee;  
INSERT INTO Employee  
VALUES (104,'Anjali',55000);
```

But Cannot Perform

```
DELETE FROM Employee;  
or  
UPDATE Employee  
SET Salary = 60000;
```

Example 3: Grant All Privileges

```
GRANT ALL PRIVILEGES  
ON Employee  
TO User1;
```

Explanation

User1 can perform:

- SELECT
- INSERT
- UPDATE
- DELETE

on the Employee table.

Common Privileges

Privilege	Purpose
SELECT	Read data
INSERT	Add new rows
UPDATE	Modify records
DELETE	Remove rows
ALL PRIVILEGES	All permissions

Example

Suppose Employee Table contains:

EmpID	Name	Salary
101	Priya	50000
102	Rahul	45000

Administrator executes:

```
GRANT SELECT
ON Employee
TO User1;
```

User1 executes:

```
SELECT *
FROM Employee;
```

Output

EmpID	Name	Salary
101	Priya	50000
102	Rahul	45000

User1 tries:

```
DELETE FROM Employee;
```

Output

ERROR: Permission denied

WITH GRANT OPTION

WITH GRANT OPTION allows a user to give the same permissions to other users.

Syntax

```
GRANT privilege_name
ON table_name
TO user_name
WITH GRANT OPTION;
```

Example

```
GRANT SELECT
ON Employee
TO User1
WITH GRANT OPTION;
```

Explanation

User1 can now grant SELECT permission to another user.

Example:

```
GRANT SELECT
ON Employee
TO User2;
```

2. REVOKE Command :

The REVOKE command is used to remove permissions that were previously granted to a user. It cancels access privileges.

Syntax

```
REVOKE privilege_name  
ON table_name  
FROM user_name;
```

Example 1

```
REVOKE SELECT  
ON Employee  
FROM User1;
```

Explanation

User1 can no longer retrieve records from the Employee table.

Example

Before REVOKE:

Administrator executes:

```
GRANT SELECT  
ON Employee  
TO User1;
```

User1 executes:

```
SELECT *  
FROM Employee;
```

Output

EmpID	Name	Salary
101	Priya	50000
102	Rahul	45000

Administrator removes permission:

```
REVOKE SELECT  
ON Employee  
FROM User1;
```

User1 tries:

```
SELECT *  
FROM Employee;
```

Output

ERROR: Access denied

Example 2: Revoke Multiple Permissions

```
REVOKE INSERT, UPDATE  
ON Employee  
FROM User1;
```

Explanation

User1 cannot:

```
INSERT INTO Employee  
VALUES(...);
```

or

```
UPDATE Employee
SET Salary = 70000;
```

Example 3: Revoke All Privileges

```
REVOKE ALL PRIVILEGES
ON Employee
FROM User1;
```

Result

User1 loses all permissions on the Employee table.

Difference Between GRANT and REVOKE

GRANT	REVOKE
Provides permissions	Removes permissions
Allows access	Restricts access
Used to authorize users	Used to cancel authorization
Increases privileges	Decreases privileges

DCL Workflow

Database Administrator

↓
GRANT Permissions

↓
User

↓ Performs Operations

↓
SELECT, INSERT,
UPDATE, DELETE

↓
REVOKE Permissions

↓
Access Removed

Advantages of DCL Commands

1.Improves Security

Only authorized users can access data.

2.Prevents Unauthorized Access

Sensitive information remains protected.

3.Supports Role-Based Access

Different users can have different permissions.

4.Provides Better Control

Administrators can easily manage database users.

5.Protects Data Integrity

Accidental changes by unauthorized users are prevented.

UNIT 18: Normalization

Normalization is the process of organizing data in a database to reduce redundancy (duplicate data) and improve data consistency.

The main goal of normalization is to divide large tables into smaller related tables and establish relationships among them.

Normalization helps:

- Eliminate duplicate data.
- Avoid data anomalies.
- Improve data integrity.
- Make database maintenance easier.
- Use storage efficiently.

Normalization is widely used in:

- Banking systems
- Hospital management systems
- Employee management systems
- E-commerce applications
- Student management systems

Why Do We Need Normalization?

Without Normalization:

- Duplicate data is stored repeatedly.
- More storage space is required.
- Updating records becomes difficult.
- Inconsistency may occur.
- Data anomalies arise.

With Normalization:

- Data redundancy is minimized.
- Storage space is saved.
- Data consistency improves.
- Updating and deleting data becomes easier.
- Database performance improves.

Example Without Normalization

Suppose we have the following Student table:

StudentID	StudentName	Course1	Course2
101	Priya	Python	SQL
102	Rahul	Java	Python
103	Anjali	Python	Django

Problems:

- Multiple courses are stored in one table.
- Difficult to add new courses.
- Duplicate information exists.
- Data maintenance becomes complex.

Normalization solves these problems by splitting the table into smaller tables.

Objectives of Normalization

Normalization aims to:

- Remove duplicate data.
- Reduce storage requirements.
- Improve consistency.
- Avoid insertion anomalies.
- Avoid deletion anomalies.
- Avoid update anomalies.
- Maintain relationships among tables.

Types of Normal Forms

Normalization is performed in stages called Normal Forms.

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. First Normal Form (1NF)

A table is said to be in First Normal Form (1NF) if:

- Each column contains atomic values.
- No repeating groups are allowed.
- Each cell contains only one value.
- Rows are uniquely identified.

Example (Before 1NF)

Student Table

StudentID	StudentName	Subjects
101	Priya	Python, SQL
102	Rahul	Java, Django

Problem:

The Subjects column contains multiple values.

This violates 1NF.

Convert into 1NF

StudentID	StudentName	Subject
101	Priya	Python
101	Priya	SQL
102	Rahul	Java
102	Rahul	Django

Now:

- Every cell contains a single value.
- No repeating groups exist.

Hence, the table satisfies First Normal Form.

Rules of 1NF

- No multivalued attributes.
- Every row should be unique.
- Each column should contain atomic values.
- Each record should have a primary key.

Advantages of 1NF

- Removes repeating groups.
- Simplifies searching.
- Improves consistency.

2. Second Normal Form (2NF)

A table is in Second Normal Form (2NF) if:

1. It is already in 1NF.
2. Every non-key attribute depends entirely on the primary key.
3. Partial dependency is removed.

Example

Course Table

StudentID	CourseID	StudentName	CourseName
101	C1	Priya	Python
101	C2	Priya	SQL
102	C1	Rahul	Python

Primary Key:

(StudentID, CourseID)

Problems:

- StudentName depends only on StudentID.
- CourseName depends only on CourseID.

This is called Partial Dependency.

Convert to 2NF

Student Table

StudentID	StudentName
101	Priya
102	Rahul

Course Table

CourseID	CourseName
C1	Python
C2	SQL

Enrollment Table

StudentID	CourseID
101	C1
101	C2
102	C1

Now every non-key column depends completely on the primary key.
Hence the tables satisfy Second Normal Form.

Advantages of 2NF

- Removes partial dependency.
- Reduces duplication.
- Improves data consistency.

3. Third Normal Form (3NF)

A table is in Third Normal Form (3NF) if:

- It is already in 2NF.
- There are no transitive dependencies.
- Non-key attributes should depend only on the primary key.

Example

Employee Table

EmpID	EmpName	DeptID	DeptName
101	Priya	D1	IT
102	Rahul	D2	HR

Primary Key:

EmpID

Problem:

DeptName depends on DeptID rather than EmpID.

This is called Transitive Dependency.

Convert into 3NF

Employee Table

EmpID	EmpName	DeptID
101	Priya	D1
102	Rahul	D2

Department Table

DeptID	DeptName
D1	IT
D2	HR

Now:

- DeptName depends only on DeptID.
- Transitive dependency is removed.

Hence the tables are in Third Normal Form.

Advantages of 3NF

- Eliminates transitive dependency.
- Reduces redundancy.
- Improves integrity.
- Simplifies maintenance.

4.Boyce-Codd Normal Form (BCNF)

BCNF is an advanced version of Third Normal Form.

A table is in BCNF if:

For every functional dependency:

$$X \rightarrow Y$$

X must be a candidate key.

Example

Course Table

Student	Course	Instructor
Priya	Python	John
Rahul	Python	John
Anjali	SQL	David

Suppose:

Course $\rightarrow$ Instructor

and

(Student, Course)

is the primary key.

Problem:

Course determines Instructor, but Course is not a candidate key.

Therefore, BCNF is violated.

Convert into BCNF

Course Table

Course	Instructor
Python	John
SQL	David

Student_Course Table

Student	Course
Priya	Python
Rahul	Python
Anjali	SQL

Now every determinant is a candidate key.

Hence the tables satisfy BCNF.

Data Anomalies

Normalization removes three types of anomalies.

1. Insertion Anomaly

Insertion anomaly occurs when certain data cannot be inserted without inserting unrelated information.

Example:

A new department cannot be added unless an employee exists.

2. Update Anomaly

Update anomaly occurs when the same information appears multiple times and requires multiple updates.

Example:

- Department name "IT" appears in many rows.
- Changing IT to Information Technology requires updating every row.
- Failure to update all rows causes inconsistency.

3. Deletion Anomaly

Deletion anomaly occurs when deleting one record accidentally removes important information.

Example:

Deleting the last employee from the HR department may also remove department details.

Functional Dependency

Functional dependency describes the relationship between attributes.

If:

$$A \rightarrow B$$

Then B depends on A.

Example

Employee Table

EmpID	Name
101	Priya
102	Rahul

Functional Dependency:

$$\text{EmpID} \rightarrow \text{Name}$$

Because each employee ID uniquely determines the employee name.

Difference Between 1NF, 2NF, 3NF, and BCNF

Normal Form	Removes
1NF	Repeating groups
2NF	Partial dependency
3NF	Transitive dependency
BCNF	Functional dependency problems

Advantages of Normalization

1.Reduces Data Redundancy

Duplicate data is minimized.

2.Improves Data Integrity

Data remains accurate and consistent.

3.Saves Storage Space

Less duplicate information is stored.

4.Simplifies Maintenance

Updating records becomes easier.

5.Prevents Data Anomalies

Insertion, deletion, and update anomalies are avoided.

Disadvantages of Normalization

- More tables are created.
- Complex queries may be required.
- More joins are needed.
- Query performance may decrease for highly normalized databases.

Summary

Normal Form	Purpose
1NF	Removes repeating groups
2NF	Removes partial dependency
3NF	Removes transitive dependency
BCNF	Removes dependency anomalies

UNIT 19: Stored Procedures

A Stored Procedure is a precompiled collection of one or more SQL statements stored inside the database and executed whenever required.

A stored procedure works like a function or program in SQL. Instead of writing the same SQL statements repeatedly, we can store them once and execute them many times.

Stored procedures improve:

- Performance
- Security
- Code reusability
- Maintainability

Stored procedures are widely used in:

- Banking Systems
- Hospital Management Systems
- Employee Management Systems
- E-Commerce Applications
- Payroll Systems

Why Do We Need Stored Procedures?

Without Stored Procedures:

- Same SQL code must be written repeatedly.
- More network traffic occurs.
- Program maintenance becomes difficult.
- Execution time increases.

With Stored Procedures:

- Code is reusable.
- Faster execution.
- Improved security.
- Easier maintenance.
- Reduced network traffic.

Example Without Stored Procedure

Suppose we want to retrieve employee details.

Every time we write:

```
SELECT *  
FROM Employee;
```

If this query is needed many times, we repeatedly write the same code.

Stored procedures solve this problem.

What is a Procedure?

A procedure is a named block of SQL statements that performs a specific task.

Once created, it can be executed multiple times using a single command.

Features of Stored Procedures

- Reusable code
- Faster execution
- Better security
- Easy maintenance
- Reduced network traffic
- Supports parameters
- Modular programming

Components of Stored Procedure

1.Procedure Name:

Unique name given to the procedure.

Example:

GetEmployee

2.Parameters:

Used to pass values into the procedure.

Example:

EmpID

3.SQL Statements :

Contains one or more SQL commands.

Example:

SELECT * FROM Employee;

4.BEGIN and END :

Represent the start and end of the procedure body.

Creating Stored Procedures

Creating a stored procedure means defining a reusable block of SQL statements and storing it permanently inside the database.

Syntax (MySQL)

```
CREATE PROCEDURE procedure_name()
BEGIN
    SQL statements;
END;
```

Example 1: Procedure Without Parameters

Employee Table

EmpID	Name	Department	Salary
101	Priya	IT	50000
102	Rahul	HR	45000
103	Anjali	Finance	55000

Create Procedure

```
CREATE PROCEDURE GetEmployees()
BEGIN
    SELECT * FROM Employee;
END;
```


Explanation

The procedure named:

GetEmployees

contains:

```
SELECT * FROM Employee;
```

Calling the Procedure

```
CALL GetEmployees();
```

Output

EmpID	Name	Department	Salary
101	Priya	IT	50000
102	Rahul	HR	45000
103	Anjali	Finance	55000

Parameters in Stored Procedures

Parameters are variables used to pass values to a stored procedure.

- They make procedures dynamic and flexible.
- Parameters are similar to function arguments in programming languages.

Types of Parameters

1. IN Parameter
2. OUT Parameter
3. INOUT Parameter

1.IN Parameter :

IN parameters are used to pass values into a stored procedure.

They are read-only inside the procedure.

Syntax

```
CREATE PROCEDURE procedure_name(IN variable datatype)
BEGIN
    SQL statements;
END;
```

Example

Employee Table

EmpID	Name	Salary
101	Priya	50000
102	Rahul	45000
103	Kiran	60000

Create Procedure

```
CREATE PROCEDURE GetEmployee(IN id INT)
BEGIN
    SELECT *
    FROM Employee
    WHERE EmpID=id;
END;
```

Execute

```
CALL GetEmployee(102);
```

Output

EmpID	Name	Salary
102	Rahul	45000

Explanation

The value:

102

is passed into the procedure and used to retrieve the corresponding employee.

OUT Parameter

OUT parameters return values from a stored procedure.

They work like return values in functions.

Example

Employee Table

EmpID	Salary
101	50000
102	45000
103	60000

Create Procedure

```
CREATE PROCEDURE GetSalary(  
  IN id INT,  
  OUT sal INT)  
BEGIN  
  SELECT Salary  
  INTO sal  
  FROM Employee  
  WHERE EmpID=id;  
END;
```

Execute

```
CALL GetSalary(101,@salary);
```

Display result:

```
SELECT @salary;
```

Output

50000

INOUT Parameter

INOUT parameters act as both input and output parameters.

They receive a value and also return a modified value.

Example

```
CREATE PROCEDURE IncreaseSalary(  
  INOUT amount INT)  
BEGIN  
  SET amount = amount + 5000;  
END;
```

Execute:

```
SET @salary=50000;  
CALL IncreaseSalary(@salary);  
SELECT @salary;
```

Output

55000

Calling Stored Procedures

Calling a procedure means executing the procedure using the CALL statement.

Syntax

```
CALL procedure_name();
```

Example

```
CALL GetEmployees();
```

Procedure with INSERT Statement

Example

```
CREATE PROCEDURE AddEmployee()  
BEGIN  
    INSERT INTO Employee  
    VALUES  
    (104,'Vijay','IT',48000);  
END;
```

Execute:

```
CALL AddEmployee();
```

Output

EmpID	Name	Department	Salary
101	Priya	IT	50000
102	Rahul	HR	45000
103	Anjali	Finance	55000
104	Vijay	IT	48000

Procedure with UPDATE Statement

Example

```
CREATE PROCEDURE UpdateSalary()  
BEGIN  
    UPDATE Employee  
    SET Salary=60000  
    WHERE EmpID=101;  
END;
```

Execute:

```
CALL UpdateSalary();
```

Output

EmpID	Salary
101	60000

Procedure with DELETE Statement

Example

```
CREATE PROCEDURE DeleteEmployee()  
BEGIN  
    DELETE FROM Employee  
    WHERE EmpID=103;  
END;
```

Execute:

```
CALL DeleteEmployee();
```

Output

EmpID	Name
101	Priya
102	Rahul

Altering Stored Procedures

Altering means modifying an existing procedure.

Syntax

```
ALTER PROCEDURE procedure_name  
MODIFY ...
```

Example:

Suppose we want to add more statements to:

```
GetEmployees
```

The procedure can be altered accordingly.

Dropping Stored Procedures :

DROP PROCEDURE removes an existing stored procedure from the database.

Syntax

```
DROP PROCEDURE procedure_name;
```

Example

```
DROP PROCEDURE GetEmployees;
```

Explanation

The procedure is deleted permanently.

Advantages of Stored Procedures

1. Code Reusability

Same procedure can be executed many times.

2. Faster Execution

Stored procedures are precompiled, so execution is faster.

3. Improved Security

Users can execute procedures without directly accessing tables.

4. Reduced Network Traffic

Only procedure calls are sent to the server.

5. Easy Maintenance

Changes are made in one place.

6. Modular Programming

Large applications can be divided into smaller procedures.

Disadvantages of Stored Procedures

- More memory is required.
- Debugging can be difficult.
- Database dependency increases.
- Complex procedures are harder to maintain.

Difference Between Function and Stored Procedure

Function	Stored Procedure
Returns a value	May or may not return a value
Used inside SQL statements	Executed using CALL statement
Usually used for calculations	Used for performing tasks
Cannot execute COMMIT or ROLLBACK	Can execute transactions

Summary

Concept	Purpose
CREATE PROCEDURE	Creates a stored procedure
IN Parameter	Passes values into procedure
OUT Parameter	Returns values from procedure
INOUT Parameter	Acts as input and output
CALL	Executes the procedure
DROP PROCEDURE	Deletes a procedure

UNIT 20: Triggers

A Trigger is a special type of stored program that automatically executes whenever a specified event occurs on a table.

Triggers are activated automatically by database operations such as:

- INSERT
- UPDATE
- DELETE

Unlike stored procedures, triggers do not need to be called explicitly. They are executed automatically when the corresponding event occurs.

Triggers are widely used in:

- Banking Systems
- Hospital Management Systems
- Employee Management Systems
- E-Commerce Applications
- Auditing Systems

Why Do We Need Triggers?

Without Triggers:

- Developers must manually perform additional tasks.
- Data validation becomes difficult.
- Audit logs need to be maintained separately.
- Data inconsistency may occur.

With Triggers:

- Operations are performed automatically.
- Data integrity is maintained.
- Audit information is generated automatically.
- Human effort is reduced.

Real-Life Example

Suppose money is deposited into a bank account.

Whenever the balance changes, the transaction details should automatically be stored in a transaction history table.

Instead of writing separate SQL statements every time, a trigger performs this task automatically.

Features of Triggers

- Execute automatically.
- Improve data consistency.
- Provide security.
- Maintain audit logs.
- Support validation.
- Reduce manual coding.
- Ensure business rules are followed.

Components of a Trigger

1.Trigger Name

The name used to identify the trigger.

Example:

trg_before_insert

2.Trigger Event :

The operation that activates the trigger.

Events include:

- INSERT
- UPDATE
- DELETE

3.Trigger Timing :

Specifies when the trigger executes.

Types:

- BEFORE
- AFTER

4.Trigger Body :

Contains SQL statements executed automatically.

Types of Triggers

Based on timing:

1. BEFORE Trigger
2. AFTER Trigger

Based on events:

1. INSERT Trigger
2. UPDATE Trigger
3. DELETE Trigger

Syntax of Trigger

```
CREATE TRIGGER trigger_name
BEFORE/AFTER INSERT/UPDATE/DELETE
ON table_name
FOR EACH ROW
BEGIN
    SQL statements;
END;
```

1. BEFORE INSERT Trigger

A BEFORE INSERT trigger executes before a new row is inserted into a table.

It is mainly used for:

- Validation
- Setting default values
- Preventing invalid data

Example

Employee Table

EmpID	Name	Salary
-------	------	--------

Create Trigger :

```
CREATE TRIGGER trg_before_insert
BEFORE INSERT
ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SET NEW.Salary = 0;
    END IF;
END;
```

Output

EmpID	Name	Salary
101	Priya	0

2. AFTER INSERT Trigger :

An AFTER INSERT trigger executes after a record has been inserted into a table.

It is commonly used for:

- Audit logs
- Transaction history
- Notifications

Example

Employee Table

EmpID	Name
101	Priya

Audit Table

Message

Create Trigger :

```
CREATE TRIGGER trg_after_insert
AFTER INSERT
ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog
    VALUES(CONCAT('Employee Added : ',NEW.Name));
END;
```

Insert Record :

```
INSERT INTO Employee
VALUES(102,'Rahul');
```


Employee Table

EmpID Name

101 Priya

102 Rahul

AuditLog Table

Message

Employee Added : Rahul

Explanation

The trigger automatically stores the information in AuditLog.

3. UPDATE Trigger :

An UPDATE trigger executes automatically whenever existing records are modified.

It is useful for:

- Tracking changes
- Maintaining logs
- Security purposes

Example

Employee Table

EmpID Salary

101 50000

Create Trigger :

```
CREATE TRIGGER trg_update_salary
AFTER UPDATE
ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO SalaryLog
    VALUES(
        OLD.Salary,
        NEW.Salary
    );
END;
```

Update Salary

```
UPDATE Employee
SET Salary=60000
WHERE EmpID=101;
```

SalaryLog Table

OldSalary NewSalary

50000 60000

Explanation

The trigger records both old and new salary values.

OLD and NEW Keywords

1.OLD

Represents the value before modification.

Example:

OLD.Salary = 50000

2.NEW

Represents the updated value.

Example:

NEW.Salary = 60000

4. DELETE Trigger :

A DELETE trigger executes automatically whenever records are deleted.

It is mainly used for:

- Backup purposes.
- Maintaining deleted records history.
- Auditing.

Example

Employee Table

EmpID	Name
101	Priya
102	Rahul

DeletedEmployee Table

EmpID	Name
-------	------

Create Trigger :

```
CREATE TRIGGER trg_delete_employee
BEFORE DELETE
ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO DeletedEmployee
    VALUES(
        OLD.EmpID,
        OLD.Name
    );
END;
```

Delete Record

```
DELETE FROM Employee
WHERE EmpID=102;
```

Employee Table

EmpID Name

101 Priya

DeletedEmployee Table

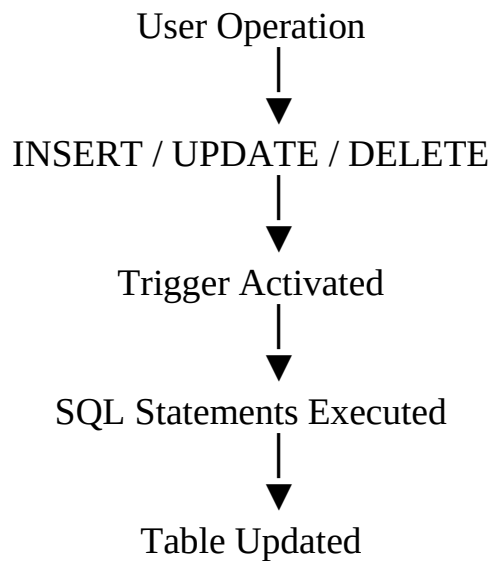
EmpID Name

102 Rahul

Explanation

Before deleting the record, the trigger stores its information in another table.

Trigger Flow :



Difference Between BEFORE and AFTER Triggers

BEFORE Trigger	AFTER Trigger
Executes before an event	Executes after an event
Used for validation	Used for logging
Can modify values	Usually records information
Prevents invalid data	Maintains history

Creating Trigger Example :

```
CREATE TRIGGER trg_insert
AFTER INSERT
ON Student
FOR EACH ROW
BEGIN
    INSERT INTO LogTable
    VALUES('New Student Added');
END;
```

Dropping a Trigger :

DROP TRIGGER removes an existing trigger from the database.

Syntax

```
DROP TRIGGER trigger_name;
```

Example

```
DROP TRIGGER trg_insert;
```

Explanation

The trigger is permanently deleted from the database.

Advantages of Triggers

1. Automatic Execution

No need to execute manually.

2. Data Integrity

Ensures consistent and valid data.

3. Audit Support

Maintains history automatically.

4. Security

Restricts invalid operations.

5. Reduces Application Code

Less coding is required in programs.

6. Enforces Business Rules

Important rules are maintained automatically.

Disadvantages of Triggers

- Difficult to debug.
- Hidden execution may confuse developers.
- Too many triggers can affect performance.
- Complex triggers increase maintenance.

Difference Between Stored Procedure and Trigger

Stored Procedure	Trigger
Executed manually	Executed automatically
Called using CALL statement	Activated by events
Can accept parameters	No parameters
Performs specific tasks	Responds to events
User controls execution	Database controls execution

Summary

Trigger Type	Purpose
BEFORE INSERT	Validate data before insertion
AFTER INSERT	Maintain logs after insertion
UPDATE Trigger	Track modifications
DELETE Trigger	Backup deleted records

UNIT 21: Common Table Expressions (CTE)

A Common Table Expression (CTE) is a temporary named result set created inside a SQL query using the WITH clause. It helps simplify complex queries and improves readability.

A CTE exists only during the execution of a query and is not stored permanently in the database like a table or view.

CTEs are commonly used in:

- Data analysis
- Reporting systems
- Employee hierarchies
- Financial applications
- Recursive queries
- Complex SQL statements

Why Do We Need CTE?

Without CTE:

- Complex queries become difficult to understand.
- Nested subqueries become lengthy.
- Query maintenance becomes harder.
- Code readability decreases.

With CTE:

- Queries become easier to read.
- Complex SQL statements are simplified.
- Code becomes reusable within a query.
- Maintenance becomes easier.

Features of CTE

- Temporary result set.
- Improves query readability.
- Simplifies nested queries.
- Supports recursive queries.
- Exists only during query execution.
- Makes SQL code easier to maintain.

Syntax of CTE

```
WITH cte_name AS
(
    SELECT column_name
    FROM table_name
)
```

```
SELECT *  
FROM cte_name;
```

Components of a CTE

1. WITH Clause :

Starts the Common Table Expression.

Example:

```
WITH EmployeeCTE AS
```

2. CTE Name :

Represents the temporary table name.

Example:

```
EmployeeCTE
```

3. Query Definition :

Contains the SQL query whose result will be stored temporarily.

Example:

```
SELECT *  
FROM Employee
```

4. Main Query :

Uses the temporary result set.

Example:

```
SELECT *  
FROM EmployeeCTE;
```

1.WITH Clause :

The WITH clause is used to create a temporary result set that can be referenced within the main query.

Syntax

```
WITH cte_name AS  
(  
    query  
)  
main_query;
```

Example 1: Simple CTE

Employee Table

EmpID	Name	Salary
101	Priya	50000
102	Rahul	65000
103	Kiran	45000

Query

```
WITH EmployeeCTE AS  
(  
    SELECT *  
    FROM Employee  
)
```

```
SELECT *
FROM EmployeeCTE;
```

Output

EmpID	Name	Salary
101	Priya	50000
102	Rahul	65000
103	Kiran	45000

Explanation

The result of:

```
SELECT *
FROM Employee
```

is temporarily stored inside:

```
EmployeeCTE
```

The main query retrieves data from this temporary table.

Example 2: Employees with Salary Greater Than 50000

Employee Table

EmpID	Name	Salary
101	Priya	50000
102	Rahul	65000
103	Kiran	45000
104	Anjali	70000

Query

```
WITH HighSalary AS
(
    SELECT *
    FROM Employee
    WHERE Salary > 50000
)
SELECT *
FROM HighSalary;
```

Output

EmpID	Name	Salary
102	Rahul	65000
104	Anjali	70000

Multiple CTEs

SQL allows multiple CTEs to be defined in the same query.

Syntax

```
WITH CTE1 AS
(
    query1
),
```

```

CTE2 AS
(
    query2
)
SELECT *
FROM CTE1, CTE2;

```

Example

```

WITH ITEmployees AS
(
    SELECT *
    FROM Employee
    WHERE Department='IT'
),
HighSalary AS
(
    SELECT *
    FROM Employee
    WHERE Salary > 50000
)
SELECT *
FROM ITEmployees;

```

2.Recursive CTE :

A Recursive CTE is a CTE that references itself.

It repeatedly executes until a specified condition becomes false.

Recursive CTEs are useful for:

- Hierarchical data
- Tree structures
- Organization charts
- Parent-child relationships
- Generating number sequences

Structure of Recursive CTE

A recursive CTE contains two parts:

Anchor Query

Provides the initial result.

Recursive Query

Repeatedly references itself.

Syntax

```

WITH RECURSIVE cte_name AS
(
    anchor_query
    UNION ALL
    recursive_query
)

```



```
SELECT *  
FROM cte_name;
```

Example 1: Generate Numbers from 1 to 5

```
WITH RECURSIVE Numbers AS
```

```
(  
    SELECT 1  
    UNION ALL  
    SELECT 1 + 1  
)
```

```
SELECT *  
FROM Numbers;
```

A better version:

```
WITH RECURSIVE Numbers AS
```

```
(  
    SELECT 1 AS num
```

```
    UNION ALL
```

```
    SELECT num + 1  
    FROM Numbers  
    WHERE num < 5  
)
```

```
SELECT *  
FROM Numbers;
```

Output

```
num  
1  
2  
3  
4  
5
```

Explanation

Anchor Query

```
SELECT 1 AS num
```

Produces:

```
1
```

Recursive Query

```
SELECT num+1  
FROM Numbers  
WHERE num<5
```

Produces:

2
3
4
5

Example 2: Employee Hierarchy

Employee Table

EmpID	Name	ManagerID
1	CEO	NULL
2	Priya	1
3	Rahul	2
4	Kiran	2

Query

```
WITH RECURSIVE EmployeeTree AS
(
    SELECT EmpID,
           Name,
           ManagerID
    FROM Employee
    WHERE ManagerID IS NULL
    UNION ALL
    SELECT E.EmpID,
           E.Name,
           E.ManagerID
    FROM Employee E
    INNER JOIN EmployeeTree ET
    ON E.ManagerID=ET.EmpID
)
SELECT *
FROM EmployeeTree;
```

Output

EmpID	Name	ManagerID
1	CEO	NULL
2	Priya	1
3	Rahul	2
4	Kiran	2

Applications of Recursive CTE

1.Organization Charts

Represent employee-manager relationships.

2.Folder Structures

Display parent and child folders.

3.Family Trees

Maintain ancestor information.

4.Category Hierarchies

Used in e-commerce websites.

5.Generating Sequences

Generate numbers and dates.

Difference Between CTE and Subquery

CTE	Subquery
Improves readability	Can become difficult to read
Reusable within query	Not reusable
Supports recursion	Does not support recursion
Easy maintenance	Complex maintenance
Temporary result set	Nested query

Difference Between CTE and View

CTE	View
Temporary	Permanent
Exists during execution only	Stored permanently
Created using WITH	Created using CREATE VIEW
Supports recursion	Limited recursion
Not stored in database	Stored in database

Advantages of CTE

1.Improved Readability

Queries become easy to understand.

2.Simplifies Complex Queries

Reduces nested subqueries.

3.Better Maintenance

SQL code becomes easier to modify.

4.Supports Recursion

Handles hierarchical data efficiently.

5.Reusability

Can be referenced multiple times in a query.

Disadvantages of CTE

- Exists only temporarily.
- Large recursive queries may affect performance.
- Cannot be indexed.
- Not suitable for storing permanent data.

Summary

Concept	Purpose
WITH Clause	Creates temporary result set
CTE	Simplifies complex queries
Recursive CTE	Handles hierarchical data
Anchor Query	Starting point of recursion
Recursive Query	Repeats until condition becomes false

UNIT 22: Window Functions

Window Functions are special SQL functions that perform calculations across a set of rows related to the current row without grouping the rows into a single result.

Unlike aggregate functions, window functions do not reduce the number of rows. They return a value for each row while considering a group of rows called a window.

Window functions are widely used in:

- Data analysis
- Reporting
- Ranking systems
- Salary comparisons
- Trend analysis
- Business intelligence
- Leaderboards

Why Do We Need Window Functions?

Without Window Functions:

- Complex queries are required.
- Self joins become complicated.
- Ranking calculations are difficult.
- Previous and next row comparisons become harder.

With Window Functions:

- Queries become simpler.
- Ranking becomes easy.
- Previous and next row values can be accessed.
- Analytical calculations become efficient.

Features of Window Functions

- Operate on multiple rows.
- Do not collapse rows like GROUP BY.
- Useful for ranking.
- Compare current row with other rows.
- Support partitioning of data.
- Useful in reporting and analytics.

Syntax of Window Function

```
function_name()  
OVER(  
PARTITION BY column_name  
ORDER BY column_name  
)
```

Components of Window Function

1.Function

Performs calculation.

Examples:

- ROW_NUMBER()
- RANK()
- DENSE_RANK()
- LEAD()
- LAG()

2.OVER()

Defines the window on which the function works.

Example:

ROW_NUMBER() OVER()

3.PARTITION BY

Divides rows into groups.

Example:

PARTITION BY Department

4.ORDER BY

Determines the sequence of rows.

Example:

ORDER BY Salary DESC

Sample Employee Table

EmpID	Name	Department	Salary
101	Priya	IT	70000
102	Rahul	HR	60000
103	Kiran	IT	80000
104	Anjali	HR	60000
105	Amit	IT	90000

1. ROW_NUMBER()

ROW_NUMBER() assigns a unique number to each row.

Even if two rows contain the same value, each row receives a different number.

Syntax

```
ROW_NUMBER()  
OVER(ORDER BY column_name)
```

Example

```
SELECT Name,  
Salary,  
ROW_NUMBER()  
OVER(ORDER BY Salary DESC) AS RowNum  
FROM Employee;
```

Output

Name	Salary	RowNum
Amit	90000	1
Kiran	80000	2
Priya	70000	3
Rahul	60000	4
Anjali	60000	5

Explanation

- Rows are sorted by salary in descending order.
- Each row receives a unique number.
- Even though Rahul and Anjali have the same salary, they get different row numbers.

2. RANK()

RANK() assigns rankings to rows.

- Rows having the same value receive the same rank.
- After duplicate ranks, gaps appear in the ranking sequence.

Syntax

```
RANK()  
OVER(ORDER BY column_name)
```

Example

```
SELECT Name,  
Salary,  
RANK()  
OVER(ORDER BY Salary DESC) AS RankNo  
FROM Employee;
```

Output

Name	Salary	RankNo
Amit	90000	1
Kiran	80000	2
Priya	70000	3
Rahul	60000	4
Anjali	60000	4

Suppose another employee with salary 50000 exists.
The next rank becomes:

6

Rank 5 is skipped.

Explanation

- Duplicate values receive the same rank.
- Gaps occur after duplicate ranks.

3. DENSE_RANK()

DENSE_RANK() assigns ranks without skipping numbers.

Rows with equal values receive the same rank, but no gaps are created.

Syntax

```
DENSE_RANK()  
OVER(ORDER BY column_name)
```

Example

```
SELECT Name,  
Salary,  
DENSE_RANK()  
OVER(ORDER BY Salary DESC) AS DenseRank  
FROM Employee;
```

Output

Name	Salary	DenseRank
Amit	90000	1
Kiran	80000	2
Priya	70000	3
Rahul	60000	4
Anjali	60000	4

If another employee has salary 50000:

Next rank becomes:

5

(No rank is skipped.)

Difference Between ROW_NUMBER(), RANK(), and DENSE_RANK()

Function	Duplicate Values	Skipped Rank
ROW_NUMBER()	No	No
RANK()	Yes	Yes
DENSE_RANK()	Yes	No

Example

Values:

```
100  
90  
90  
80
```

ROW_NUMBER()

```
1  
2  
3  
4
```

RANK()

```
1  
2  
2  
4
```

DENSE_RANK()

1

2

2

3

4. LEAD()

LEAD() returns the value from the next row.

It is used to compare the current row with the following row.

Syntax

```
LEAD(column_name)
OVER(ORDER BY column_name)
```

Example

```
SELECT Name,
Salary,
LEAD(Salary)
OVER(ORDER BY Salary) AS NextSalary
FROM Employee;
```

Output

Name	Salary	NextSalary
Rahul	60000	60000
Anjali	60000	70000
Priya	70000	80000
Kiran	80000	90000
Amit	90000	NULL

Explanation

LEAD() fetches the next row's value.

For the last row, NULL is returned because no next row exists.

5. LAG()

LAG() returns the value from the previous row.

It is used to compare the current row with the preceding row.

Syntax

```
LAG(column_name)
OVER(ORDER BY column_name)
```

Example

```
SELECT Name,
Salary,
LAG(Salary)
OVER(ORDER BY Salary) AS PreviousSalary
FROM Employee;
```


Output

Name	Salary	PreviousSalary
Rahul	60000	NULL
Anjali	60000	60000
Priya	70000	60000
Kiran	80000	70000
Amit	90000	80000

Explanation

LAG() retrieves the previous row value.

The first row returns NULL because no previous row exists.

PARTITION BY

PARTITION BY divides rows into groups.

Window functions are applied separately to each group.

Example

```
SELECT Name,  
       Department,  
       Salary,  
       ROW_NUMBER()  
       OVER(  
         PARTITION BY Department  
         ORDER BY Salary DESC  
       ) AS RankNo  
FROM Employee;
```

Output

IT Department

Name	Salary	RankNo
Amit	90000	1
Kiran	80000	2
Priya	70000	3

HR Department

Name	Salary	RankNo
Rahul	60000	1
Anjali	60000	2

Advantages of Window Functions

1.Simplify Complex Queries

Reduce the need for self joins.

2.Better Performance

Efficient analytical calculations.

3.Easy Ranking

Generate ranks quickly.

4.Trend Analysis

Compare current row with previous and next rows.

5.Reporting

Useful for dashboards and business intelligence.

Disadvantages of Window Functions

- Complex for beginners.
- Large datasets may affect performance.
- Require ORDER BY for meaningful results.
- Supported only in modern databases.

Summary

Function	Purpose
ROW_NUMBER()	Assign unique row numbers
RANK()	Ranking with gaps
DENSE_RANK()	Ranking without gaps
LEAD()	Access next row value
LAG()	Access previous row value
PARTITION BY	Divide rows into groups
OVER()	Define window for calculation